PROJETO 2 DE 5 · PORTFÓLIO PROFISSIONAL

MarketGo

Compras com entrega: catálogo, carrinho, checkout e pedidos

Guia de construção passo a passo

Ao final da leitura, o aplicativo estará totalmente funcional.

Disciplina Programação para Dispositivos Móveis

Professor Prof. Me. Bruno Assunção Dias

Instituição IESB

Edição Setembro de 2026

Tecnologias React Native · Expo · TypeScript · Supabase (RLS e funções SQL) · TanStack Query · Zod · Jest

Sumário

Antes de começar
O que será construído, decisões de segurança, tecnologias e como usar o guia **Etapa 1 — Preparar o ambiente**
Node.js, VS Code, Expo Go e contas necessárias
Etapa 2 — Criar o projeto e configurar as ferramentas Criar o projeto Expo, instalar bibliotecas e configurar
Etapa 3 — Configurar o banco no Supabase
Tabelas, RLS, funções de pedido e dados de exemplo
Etapa 4 — Testar o banco no seu computador
Testar o banco no computador com PGlite (13 testes)
Etapa 5 — Fundações: tipos, tema e conexão
Tipos, tema, constantes, cliente Supabase e utilitários
Etapa 6 — Regras de negócio e testes automatizados Carrinho, catálogo, pedidos e checkout

como funções puras (37 testes) **Etapa 7 — Camada de dados: serviços e hooks**
Serviços, RPC do pedido e hooks com TanStack Query
Etapa 8 — Design system: componentes de interface Componentes de interface: lojas, produtos, quantidade e linha do tempo **Etapa 9 — Sessão e carrinho: os contextos**
Sessão do usuário e carrinho salvo no aparelho
Etapa 10 — Navegação e autenticação
Login, cadastro, abas e rotas protegidas
Etapa 11 — Catálogo: lojas, cardápio e produto Lojas com busca, cardápio, detalhe do produto e barra do carrinho **Etapa 12 — Carrinho e checkout**
Carrinho, pedido mínimo e checkout com validação
Etapa 13 — Pedidos e acompanhamento
Histórico, linha do tempo, cancelamento e pedir novamente
Etapa 14 — Qualidade: verificações e roteiro de testes Verificações automáticas (50 testes) e roteiro de 18 cenários **Etapa 15 — Portfólio: GitHub, README e apresentação** GitHub, README, build de teste e apresentação
Apêndice A — Árvore completa de arquivos
Árvore completa de arquivos
Apêndice B — Solução de problemas
Sintomas, causas e soluções
Apêndice C — Desafios de evolução
Ideias para evoluir o projeto
Apêndice D — Cronograma e avaliação
Cronograma sugerido e critérios de avaliação

Dica: os títulos acima são links. Toque ou clique para ir direto à etapa.

Antes de começar

Este guia leva você, do zero, à construção completa do **MarketGo**: um aplicativo de compras com entrega. O usuário escolhe uma loja, monta o carrinho, finaliza o pedido e acompanha cada etapa até a entrega. Cada etapa termina com um **ponto de verificação**. Só avance quando ele estiver funcionando.

Ao chegar ao fim da leitura, o aplicativo estará **totalmente funcional**, com testes automatizados no app **e no banco de dados**, e publicado no seu GitHub. O pagamento é **simulado**: nenhum valor é cobrado e nenhum pedido é entregue de verdade.

O que você vai construir

- **Conta de usuário:** cadastro, login, logout e sessão persistente.
- **Catálogo:** lojas com busca, filtro por categoria e aviso de loja fechada; cardápio por seção e detalhe do produto.
- **Carrinho:** salvo no aparelho, com a regra “uma loja por vez” e pedido mínimo.
- **Checkout:** endereço, forma de pagamento e troco, com validação.
- **Pedidos:** histórico, linha do tempo que se atualiza sozinha, cancelamento e “Pedir novamente”.
- **Segurança:** o total do pedido é calculado **no servidor**, e o app nem consegue escrever pedidos direto no banco.

Telas do aplicativo

Tela	Rota	O que faz
Login e Cadastro	/login, /register	Entrar e criar conta.
Lojas	/	Lista de lojas com busca e categorias.
Loja	/store/[id]	Cardápio por seção e barra do carrinho.
Produto	/product/[id]	Detalhe, quantidade e adicionar ao carrinho.
Carrinho	/cart	Itens, quantidades, totais e pedido mínimo.
Finalizar pedido	/checkout	Endereço, pagamento e confirmação.
Pedidos	/orders	Histórico de pedidos.
Acompanhar pedido	/order/[id]	Linha do tempo, itens, cancelar e pedir novamente.
Perfil	/profile	Dados da conta e sair.

A decisão de arquitetura mais importante

Em apps de compra, **nunca** confie no que o aplicativo envia. Quem está no celular pode alterar o preço de um item antes de enviar. Por isso, o MarketGo funciona assim:

Passo	Quem faz	O que acontece
1	App	Monta o carrinho e envia **apenas ids e quantidades** (nenhum preço).

2	Função `create_order`	Roda no banco. Confere se a loja está aberta, se cada produto existe, está ativo e é daquela loja.
3	Função `create_order`	Calcula o subtotal com os **preços do banco**, soma a entrega, confere o pedido mínimo e o troco.
4	Função `create_order`	Grava o pedido e **copia** nome e preço de cada item (snapshot). Devolve o id.
5	App	O usuário só **lê** os próprios pedidos. Não existe permissão de escrita direta.

BOM SABER

Regra de ouro: o app pede, o banco decide. Toda regra que envolve dinheiro fica no servidor, e o app apenas apresenta o resultado.

Tecnologias e por que o mercado as valoriza

Tecnologia	Papel no projeto	Valor no mercado
React Native + Expo	App nativo para iOS e Android	Base das vagas mobile multiplataforma
TypeScript	Tipagem em todo o código	Exigência na maioria das vagas
Postgres com RLS e funções SQL	Regras de negócio e segurança dentro do banco	Habilidade rara e muito valorizada
PGlite	Testar o banco no computador, sem Docker	Testes de banco de verdade, rápidos e reproduzíveis
TanStack Query	Cache e atualização automática dos pedidos	Padrão para dados remotos em React
React Hook Form + Zod	Checkout com validação tipada	Combinação muito usada em produção
Jest	Testes das regras do carrinho e do catálogo	Qualidade comprovada no repositório

Como usar este guia

- Blocos escuros com o título **Terminal** são comandos para o terminal do VS Code (atalho Ctrl + crase).
- Blocos **Código N** são arquivos completos. O caminho aparece no cabeçalho: crie o arquivo exatamente naquele lugar e cole o conteúdo inteiro.
- Caixas verdes **Ponto de verificação** dizem o que você deve ver antes de seguir. • Caixas amarelas **Atenção** avisam sobre erros comuns.
- Se você já fez o **FinTrack**, vai reconhecer a base (login, tema, componentes). Ela está aqui completa, para o guia ser independente.

BOM SABER

Versões usadas: Expo SDK 57, React Native 0.86, React 19.2 e TypeScript 6. O código deste guia foi conferido com verificação de tipos sem erros, 37 testes do app e 13 testes do banco passando, empacotamento para Android e um teste do fluxo de compra rodando a interface (lojas, carrinho, checkout, pedido e status) com um servidor simulado.

Etapa 1 — Preparar o ambiente

Instale as ferramentas abaixo antes de escrever qualquer código.

Ferramenta	Para quê	Como conferir
Node.js (versão LTS)	Executa o Expo, o npm e os testes do banco	`node -v`
VS Code	Editor do projeto	Abrir o programa
Git	Versionamento e GitHub	`git --version`
Expo Go (celular)	Abre o app durante o desenvolvimento	Instalar pela loja de aplicativos
Conta no Supabase	Banco de dados e autenticação	supabase.com
Conta no GitHub	Publicar o portfólio	github.com

Extensões do VS Code

- **Expo Tools:** autocompletar do `app.json` e apoio à depuração.
- **ESLint** e **Prettier - Code formatter:** qualidade e padronização do código.
- **PostgreSQL** ou **SQL Tools** (opcional): realce e formatação dos arquivos `.sql`.

Em **Settings**, ative *Format On Save* e escolha o Prettier como formatador padrão. Confira as versões:

Terminal bash

```
node -v
npm -v
git --version
```

PONTO DE VERIFICAÇÃO

- `node -v` mostra a versão **22 ou superior**. O teste do banco usa o executor de testes nativo do Node, que exige uma versão recente.
- Expo Go instalado no celular e contas do Supabase e do GitHub criadas.
- Celular e computador na **mesma rede Wi-Fi**.

ATENÇÃO

Redes de faculdade costumam bloquear a conexão entre o celular e o computador. Se o QR Code não abrir o app, use `npx expo start --tunnel` (veja o Apêndice B).

Etapa 2 — Criar o projeto e configurar as ferramentas

2.1 Criar o projeto

Terminal bash

```
npx create-expo-app@latest marketgo --template blank-typescript
cd marketgo
code .
```

Daqui em diante, os comandos rodam no terminal integrado do VS Code, dentro da pasta `marketgo`. Apague os arquivos `App.tsx` e `index.ts` da raiz: com o Expo Router, o ponto de entrada passa a ser a pasta `app/`.

2.2 Instalar as bibliotecas

Primeiro, as bibliotecas do ecossistema Expo. O comando `expo install` escolhe a versão compatível com o seu SDK.

Terminal bash

```
npx expo install expo-router react-native-safe-area-context react-native-screens npx expo install
expo-linking expo-constants expo-status-bar react-dom react-native-web npx expo install
@expo/vector-icons @react-native-async-storage/async-storage
```

Pacote	Função
expo-router	Navegação baseada em arquivos
react-native-safe-area-context, react native-screens	Áreas seguras e telas nativas (exigidos pelo Router)
expo-linking, expo-constants, expo-status bar	Links, leitura do app.json e barra de status
react-dom, react-native-web	Dependências de par do Expo Router. Evitam erro na instalação
@expo/vector-icons	Ícones (conjunto Ionicons)
@react-native-async-storage/async-storage	Guarda a sessão e o **carrinho** no aparelho

Agora as demais bibliotecas:

Terminal bash

```
npm install @supabase/supabase-js @tanstack/react-query react-native-url-polyfill npm install
react-hook-form zod @hookform/resolvers
```

Pacote	Função
@supabase/supabase-js	Cliente de autenticação, banco e funções
@tanstack/react-query	Cache e atualização automática dos dados
react-hook-form, zod, @hookform/resolvers	Formulários e validação tipada

Pacote	Função
react-native-url-polyfill	Complemento exigido pelo Supabase no React Native

Por fim, as ferramentas de desenvolvimento. O @electric-sql/pglite é um Postgres de verdade que roda dentro do Node e servirá para testar o banco:

Terminal bash

```
npm install -D jest-expo jest @types/jest prettier @electric-sql/pglite
npx expo install --fix
```

ATENÇÃO

Se o npm install acusar ERESOLVE, abra o package.json e confira se react-dom está na **mesma versão** de react, ambas sem o acento circunflexo. Ajuste e rode npm install de novo. Só em último caso use npm install --legacy-peer-deps.

2.3 Ajustar o package.json

Abra o package.json. Altere o campo main, substitua o bloco scripts e adicione o bloco jest. Mantenha dependencies e devDependencies como estão.

Código 1 · package.json (trechos) JSON

```
{
"main": "expo-router/entry",
"scripts": {
"start": "expo start",
"android": "expo start --android",
"ios": "expo start --ios",
"test": "jest && node --test \"supabase/tests/*.test.mjs\"",
"test:app": "jest",
"test:db": "node --test \"supabase/tests/*.test.mjs\"",
"typecheck": "tsc --noEmit",
"format": "prettier --write app src __tests__ supabase/tests"
},
"jest": {
"preset": "jest-expo",
"moduleNameMapper": {
"^@/(.*)$": "<rootDir>/src/$1"
}
}
}
```

- test:app roda o Jest (regras do app). test:db roda os testes do banco com o executor nativo do Node. test roda os dois.
- moduleNameMapper ensina o Jest a entender o atalho @/, que aponta para a pasta src/.

2.4 app.json

Código 2 · app.json JSON

```
{
"expo": {
"name": "MarketGo",
"slug": "marketgo",
"scheme": "marketgo",
"version": "1.0.0",
"orientation": "portrait",
"icon": "./assets/icon.png",
"userInterfaceStyle": "light",
"plugins": [
"expo-router"
],
"ios": {
"supportsTablet": true,
"bundleIdentifier": "com.seuusuario.marketgo"
},
"android": {
"package": "com.seuusuario.marketgo",
"adaptiveIcon": {
"backgroundColor": "#E6F4FE",
"foregroundImage": "./assets/android-icon-foreground.png",
"backgroundImage": "./assets/android-icon-background.png",
"monochromeImage": "./assets/android-icon-monochrome.png"
},
"predictiveBackGestureEnabled": false
}
}
}
```

- `scheme` é o nome usado em links profundos do app.
- `bundleIdentifier` e `package` identificam o app nas lojas. Troque `seuusuario` por algo único.

ATENÇÃO

Mantenha os caminhos de ícones que já existem no seu `app.json`. Se o template gerou nomes diferentes dos mostrados aqui, não altere.

2.5 tsconfig.json

Código 3 · tsconfig.json JSON

```
{
"extends": "expo/tsconfig.base",
"compilerOptions": {
"strict": true,
"types": ["jest"],
```

```
"paths": {
"@/*": ["./src/*"]
}
},
"include": ["**/*.ts", "**/*.tsx", ".expo/types/**/*.ts", "expo-env.d.ts"]
}
```

- `paths` cria o atalho `@/`, que permite importar `@/utils/cart` em vez de `../../utils/cart`.

- `types: ["jest"]` carrega os tipos do Jest. A partir do TypeScript 6, os pacotes `@types` não são carregados automaticamente.

2.6 Prettier, variáveis de ambiente e .gitignore

Código 4 · .prettierrc JSON

```
{
"singleQuote": true,
"printWidth": 90
}
```

Código 5 · .env.example Variáveis

```
EXPO_PUBLIC_SUPABASE_URL=https://SEU-PROJETO.supabase.co
EXPO_PUBLIC_SUPABASE_KEY=SUA-CHAVE-ANON-OU-PUBLISHABLE
```

Copie o `.env.example` para um arquivo chamado `.env` (na raiz). Os valores reais entram na Etapa 3. Em seguida, acrescente estas linhas ao final do `.gitignore`:

Código 6 · .gitignore (acrescente ao final) Texto

```
# Segredos locais
.env
.env.*
!.env.example
```

BOM SABER

Variáveis com o prefixo `EXPO_PUBLIC_` são embutidas no app durante o empacotamento. Por isso, só colocamos ali a URL e a chave **pública** do Supabase. Nunca use a chave `service_role` dentro de um aplicativo.

2.7 Teste de fumaça

Código 7 · app/index.tsx (temporário) TSX

```
import { Text, View } from 'react-native';

export default function Home() {
return (
<View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}> <Text>MarketGo
funcionando!</Text>
```

```
</View>
);
}
```

```
Terminal bash npx expo start
```

Escaneie o QR Code com o Expo Go (Android) ou com a câmera (iPhone).

PONTO DE VERIFICAÇÃO

- O celular mostra **MarketGo funcionando!**
- Ao salvar o arquivo, a tela atualiza sozinha (Fast Refresh). • O arquivo `expo-env.d.ts` foi criado na raiz do projeto.

ATENÇÃO

O `app/index.tsx` é **temporário**. Você vai apagá-lo no início da Etapa 10.

Etapa 3 — Configurar o banco no Supabase

Nesta etapa, o banco de dados deixa de ser um simples depósito de dados: ele passa a **aplicar as regras do negócio**. Vamos criar as tabelas, a segurança e três funções que controlam todo o ciclo do pedido.

3.1 Criar o projeto

1. Acesse supabase.com, entre com a sua conta e clique em **New project**.
2. Nome: `marketgo`. Defina uma senha forte para o banco e guarde-a (ela **não** é usada no app). 3. Região: escolha a mais próxima, por exemplo South America (São Paulo).
4. Aguarde alguns minutos até o projeto ficar ativo.

3.2 Tabelas, segurança e funções

No menu lateral, abra **SQL Editor**, crie uma consulta nova, cole o script abaixo e clique em **Run**. Guarde uma cópia no projeto, em `supabase/schema.sql`.

Código 8 · supabase/schema.sql SQL

```sql
-- MarketGo: esquema do banco (execute no SQL Editor do Supabase)

-- 1) Catálogo ------------------------------------------------------
create table public.stores (
 id uuid primary key default gen_random_uuid(),
 name text not null,
 category text not null
 check (category in ('Mercado', 'Padaria', 'Farmácia', 'Restaurante', 'Hortifrúti')), emoji text not null,
 description text not null default '',
 rating numeric(2, 1) not null default 4.5 check (rating between 0 and 5),
 delivery_fee numeric(8, 2) not null default 0 check (delivery_fee >= 0),
 eta_min int not null default 30 check (eta_min > 0),
 eta_max int not null default 45 check (eta_max >= eta_min),
 min_order numeric(8, 2) not null default 0 check (min_order >= 0),
 is_open boolean not null default true,
 created_at timestamptz not null default now()
);

create table public.products (
 id uuid primary key default gen_random_uuid(),
 store_id uuid not null references public.stores (id) on delete cascade,
 section text not null,
 name text not null,
 description text not null default '',
 price numeric(8, 2) not null check (price > 0),
 emoji text not null,
 is_active boolean not null default true
);

create index products_store_idx on public.products (store_id, section, name);

-- 2) Pedidos -------------------------------------------------------
-- Os textos da loja e dos itens são copiados no momento da compra ("snapshot"): se o preço mudar depois, o pedido antigo continua igual.
create table public.orders (
 id uuid primary key default gen_random_uuid(),
 user_id uuid not null default auth.uid()
 references auth.users (id) on delete cascade,
```



```sql
 store_id uuid not null references public.stores (id),
 store_name text not null,
 status text not null default 'received'
 check (status in ('received', 'preparing', 'out_for_delivery', 'delivered', 'cancelled')), payment_method text not null
check (payment_method in ('pix', 'card', 'cash')), change_for numeric(8, 2) check (change_for is null or change_for > 0), address_street text not null,
 address_number text not null,
 address_neighborhood text not null,
```

```
 address_complement text,
 subtotal numeric(10, 2) not null check (subtotal >= 0),
 delivery_fee numeric(8, 2) not null check (delivery_fee >= 0),
 total numeric(10, 2) not null check (total >= 0),
 created_at timestamptz not null default now(),
 updated_at timestamptz not null default now()
);

create index orders_user_idx on public.orders (user_id, created_at desc);

create table public.order_items (
 id uuid primary key default gen_random_uuid(),
 order_id uuid not null references public.orders (id) on delete cascade,  product_id uuid
references public.products (id) on delete set null,
 name text not null,
 emoji text not null,
 unit_price numeric(8, 2) not null check (unit_price > 0),
 quantity int not null check (quantity between 1 and 20),
 total_price numeric(10, 2) not null check (total_price > 0)
);

create index order_items_order_idx on public.order_items (order_id);

-- 3) Segurança: Row Level Security ---------------------------------- alter table public.stores
enable row level security;
alter table public.products enable row level security;
alter table public.orders enable row level security;
alter table public.order_items enable row level security;

create policy "Lojas: leitura para usuários logados"
 on public.stores for select to authenticated using (true);

create policy "Produtos: leitura dos ativos"
 on public.products for select to authenticated using (is_active);

create policy "Pedidos: ler os próprios"
 on public.orders for select to authenticated
 using ((select auth.uid()) = user_id);

create policy "Itens: ler os dos próprios pedidos"
 on public.order_items for select to authenticated
 using (
 exists (
 select 1 from public.orders o
 where o.id = order_id and o.user_id = (select auth.uid())
 )
 );

-- Repare: NÃO existe política de insert, update ou delete em orders e order_items. -- O app não
escreve pedidos diretamente. Só as funções abaixo escrevem, e são elas -- que calculam preços e
controlam o status.

grant select on public.stores, public.products to authenticated;
grant select on public.orders, public.order_items to authenticated;

-- 4) Funções (RPC): a única porta de escrita dos pedidos ---------------- create or replace
function public.create_order(
 p_store_id uuid,
 p_items jsonb,
```



```
 p_payment text,
 p_change_for numeric,
 p_street text,
 p_number text,
 p_neighborhood text,
 p_complement text
```

```
) returns uuid
language plpgsql
security definer
set search_path = public
as $$
declare
 v_user uuid := auth.uid();
 v_store public.stores;
 v_count int;
 v_subtotal numeric(10, 2);
 v_total numeric(10, 2);
 v_order_id uuid;
begin
 if v_user is null then
 raise exception 'Faça login para fazer um pedido.';
 end if;

 select * into v_store from public.stores where id = p_store_id;
 if not found then
 raise exception 'Loja não encontrada.';
 end if;
 if not v_store.is_open then
 raise exception 'A loja está fechada no momento.';
 end if;

 if p_payment not in ('pix', 'card', 'cash') then
 raise exception 'Forma de pagamento inválida.';
 end if;
 if coalesce(trim(p_street), '') = '' or coalesce(trim(p_number), '') = ''  or
coalesce(trim(p_neighborhood), '') = '' then
 raise exception 'Endereço de entrega incompleto.';
 end if;

 if jsonb_typeof(p_items) <> 'array' or jsonb_array_length(p_items) = 0 then  raise exception 'O
carrinho está vazio.';
 end if;

 -- Cada item precisa existir, estar ativo, ser desta loja, ter quantidade de 1 a 20  -- e não pode se repetir.
Os PREÇOS vêm do banco, nunca do app.
 select count(*) into v_count
 from jsonb_to_recordset(p_items) as i(product_id uuid, quantity int)  join
public.products p
 on p.id = i.product_id and p.store_id = p_store_id and p.is_active  where i.quantity
between 1 and 20;

 if v_count <> jsonb_array_length(p_items)
 or (select count(distinct e ->> 'product_id') from jsonb_array_elements(p_items) e)  <>
jsonb_array_length(p_items) then
 raise exception 'Há itens inválidos no carrinho. Atualize e tente de novo.';  end if;

 select sum(p.price * i.quantity) into v_subtotal
 from jsonb_to_recordset(p_items) as i(product_id uuid, quantity int)  join
public.products p on p.id = i.product_id;

 if v_subtotal < v_store.min_order then
 raise exception 'O pedido mínimo desta loja é R$ %.',
 replace(trim(to_char(v_store.min_order, 'FM999990.00')), '.', ',');  end if;

 v_total := v_subtotal + v_store.delivery_fee;
```



```
 if p_payment = 'cash' and p_change_for is not null and p_change_for < v_total then  raise exception 'O troco
deve ser para um valor maior ou igual ao total do pedido.';  end if;

 insert into public.orders (
```

```
 user_id, store_id, store_name, payment_method, change_for,
 address_street, address_number, address_neighborhood, address_complement,  subtotal,
delivery_fee, total
 ) values (
 v_user, v_store.id, v_store.name, p_payment,
 case when p_payment = 'cash' then p_change_for end,
 trim(p_street), trim(p_number), trim(p_neighborhood), nullif(trim(p_complement), ''),  v_subtotal,
v_store.delivery_fee, v_total
 ) returning id into v_order_id;

 insert into public.order_items
 (order_id, product_id, name, emoji, unit_price, quantity, total_price)  select v_order_id, p.id, p.name,
p.emoji, p.price, i.quantity, p.price * i.quantity  from jsonb_to_recordset(p_items) as i(product_id uuid,
quantity int)  join public.products p on p.id = i.product_id;

 return v_order_id;
end;
$$;

-- Modo demonstração: avança o pedido para o próximo status. Em um produto real, quem -- faz isso é a
loja ou o entregador, nunca o cliente.
create or replace function public.advance_order(p_order_id uuid)
returns text
language plpgsql
security definer
set search_path = public
as $$
declare
 v_status text;
 v_next text;
begin
 select status into v_status
 from public.orders
 where id = p_order_id and user_id = auth.uid()
 for update;

 if not found then
 raise exception 'Pedido não encontrado.';
 end if;

 v_next := case v_status
 when 'received' then 'preparing'
 when 'preparing' then 'out_for_delivery'
 when 'out_for_delivery' then 'delivered'
 end;
 if v_next is null then
 raise exception 'Este pedido não pode avançar.';
 end if;

 update public.orders set status = v_next, updated_at = now() where id = p_order_id;  return v_next;
end;
$$;

create or replace function public.cancel_order(p_order_id uuid)
returns void
language plpgsql
security definer
set search_path = public
as $$
declare
 v_status text;
begin
```



```
 select status into v_status
 from public.orders
 where id = p_order_id and user_id = auth.uid()
 for update;
```

```
 if not found then
 raise exception 'Pedido não encontrado.';
 end if;
 if v_status <> 'received' then
 raise exception 'Só é possível cancelar um pedido que ainda não entrou em preparo.';  end if;

 update public.orders set status = 'cancelled', updated_at = now() where id = p_order_id; end;
$$;

-- Por padrão, toda função pode ser executada por qualquer um. Fechamos e liberamos só -- para quem
está logado.
revoke all on function
 public.create_order(uuid, jsonb, text, numeric, text, text, text, text) from public; revoke all on function
public.advance_order(uuid) from public;
revoke all on function public.cancel_order(uuid) from public;
grant execute on function
 public.create_order(uuid, jsonb, text, numeric, text, text, text, text) to authenticated; grant execute on function
public.advance_order(uuid) to authenticated;
grant execute on function public.cancel_order(uuid) to authenticated;
```

As tabelas

- `stores` e `products`: o catálogo. Cada loja tem taxa de entrega, tempo estimado, pedido mínimo e se está aberta. Cada produto pertence a uma loja e a uma **seção** do cardápio.
- `orders` e `order_items`: os pedidos. Repare em `store_name`, `name` e `unit_price`: são **cópias** (snapshot) dos dados no momento da compra. Se o preço mudar amanhã, o pedido de hoje continua igual.
- Restrições (`check`): preço maior que zero, quantidade de 1 a 20, status e forma de pagamento só com valores válidos. O banco recusa dado inconsistente, mesmo que o app tenha um erro.

A segurança: quem lê e quem escreve

- **Leitura:** lojas e produtos ativos podem ser lidos por qualquer usuário logado. Pedidos e itens, **somente pelo dono**.
- **Escrita:** repare que **não existe política de insert, update ou delete** em `orders` e `order_items`. Com o RLS ligado e sem política, ninguém consegue escrever por essas tabelas, nem o dono.
- A única porta de escrita são as **funções**, que o app chama por RPC (`supabase.rpc`).

As funções

- **«create_order»:** valida login, loja aberta, pagamento, endereço e itens (existem, estão ativos, são da loja, quantidade de 1 a 20, sem repetição). Calcula o subtotal **com os preços do banco**, confere o pedido mínimo e o troco, grava o pedido e copia os itens.
- **«advance_order»:** avança o status na ordem certa (recebido, em preparo, saiu para entrega, entregue). Existe só para a **demonstração**: em um app real, quem faz isso é a loja.
- **«cancel_order»:** cancela apenas enquanto o pedido está “recebido”.

- `security definer` faz a função rodar com os direitos do dono (necessário para escrever nas

tabelas fechadas). Por isso, ela mesma confere o usuário com `auth.uid()` e usa `set search_path`, uma proteção padrão para funções desse tipo.

- `for update` bloqueia a linha do pedido durante a mudança de status, evitando que dois toques simultâneos avancem o pedido duas vezes.
- `revoke ... from public` e `grant execute ... to authenticated` fecham a porta para visitantes: por padrão, o Postgres deixa qualquer um executar uma função.
- As mensagens de erro já estão em português, e o app as exibe direto ao usuário.

3.3 Dados de exemplo

Em uma nova consulta, rode o script com as lojas e os produtos. Guarde uma cópia em `supabase/seed.sql`. São 6 lojas (uma delas **fechada**, para testar esse caso) e 50 produtos.

Código 9 · supabase/seed.sql SQL

```sql
-- MarketGo: dados iniciais (execute DEPOIS do schema.sql)

insert into public.stores
 (name, category, emoji, description, rating,
 delivery_fee, eta_min, eta_max, min_order, is_open)
values
 ('Mercado Bom Preço', 'Mercado', '🛒', 'Supermercado de bairro com entrega rápida.', 4.7, 5.9, 30, 45,
20.0, true),
 ('Padaria Pão Quente', 'Padaria', '🥖', 'Pães e cafés saindo do forno o dia todo.', 4.8, 3.5, 15, 25, 10.0,
true),
 ('Farmácia Vida', 'Farmácia', '💊', 'Medicamentos e higiene, sem sair de casa.', 4.6, 6.0, 25, 40,
15.0, true),
 ('Hortifrúti da Terra', 'Hortifrúti', '🥬', 'Frutas, verduras e legumes fresquinhos.', 4.5, 4.9, 30, 50, 25.0, true),
 ('Pizzaria Forno a Lenha', 'Restaurante', '🍕', 'Pizzas artesanais de massa fina.', 4.9, 7.9, 40, 60, 30.0,
true),
 ('Sushi Sakura', 'Restaurante', '🍣', 'Culinária japonesa. Volta amanhã.',
 4.4, 9.9, 45, 65, 40.0, false);

insert into public.products (store_id, section, name, description, price, emoji) select st.id, v.section,
v.name, v.description, v.price, v.emoji
from (values
 ('Hortifrúti', 'Banana prata 1 kg', '', 6.9, '🍌'),
 ('Hortifrúti', 'Maçã gala 1 kg', '', 9.9, '🍎'),
 ('Hortifrúti', 'Tomate 1 kg', '', 7.5, '🍅'),
 ('Mercearia', 'Arroz branco 5 kg', 'Tipo 1, grãos longos.', 29.9, '🍚'),
 ('Mercearia', 'Feijão carioca 1 kg', '', 8.5, '🫘'),
 ('Mercearia', 'Macarrão espaguete 500 g', '', 4.8, '🍝'),
 ('Mercearia', 'Café torrado 500 g', 'Moagem média.', 18.9, '☕'),
 ('Laticínios', 'Leite integral 1 L', '', 5.25, '🥛'),
 ('Laticínios', 'Queijo mussarela 200 g', '', 14.9, '🧀'),
 ('Laticínios', 'Manteiga 200 g', '', 11.9, '🧈'),
 ('Limpeza', 'Detergente 500 ml', '', 2.9, '🧴'),
 ('Limpeza', 'Sabão em pó 1 kg', '', 12.5, '🧼')
 ) as v(section, name, description, price, emoji)
 cross join (select id from public.stores where name = 'Mercado Bom Preço') as st union all
select st.id, v.section, v.name, v.description, v.price, v.emoji
from (values
 ('Pães', 'Pão francês (6 un)', 'Crocante por fora, macio por dentro.', 4.8, '🥖'), ('Pães', 'Pão de queijo (10
un)', '', 12.0, '🧀'),
 ('Pães', 'Pão na chapa', '', 5.5, '🥪'),
```



```sql
 ('Doces', 'Bolo de cenoura (fatia)', 'Com cobertura de chocolate.', 7.5, '🍰'), ('Bebidas', 'Café com
leite', '', 6.5, '☕'),
 ('Bebidas', 'Suco de laranja 300 ml', '', 8.0, '🧃'),
```

```
('Salgados', 'Coxinha', '', 6.5, '??'),
('Salgados', 'Empada de frango', '', 6.0, '??')
) as v(section, name, description, price, emoji)
cross join (select id from public.stores where name = 'Padaria Pão Quente') as st union all
select st.id, v.section, v.name, v.description, v.price, v.emoji
from (values
('Analgésicos', 'Dipirona 500 mg (10 comp.)', '', 9.9, '??'),
('Analgésicos', 'Paracetamol 750 mg (10 comp.)', '', 11.5, '??'),  ('Higiene', 'Sabonete
90 g', '', 2.5, '??'),
('Higiene', 'Creme dental 90 g', '', 5.9, '??'),
('Higiene', 'Escova de dentes', '', 7.9, '??'),
('Higiene', 'Protetor solar FPS 50', '', 39.9, '??'),
('Cuidados', 'Álcool em gel 500 ml', '', 9.9, '??'),
('Cuidados', 'Curativo (20 un)', '', 8.5, '??')
) as v(section, name, description, price, emoji)
cross join (select id from public.stores where name = 'Farmácia Vida') as st union all
select st.id, v.section, v.name, v.description, v.price, v.emoji
from (values
('Frutas', 'Laranja 1 kg', '', 5.9, '??'),
('Frutas', 'Mamão papaia', '', 6.5, '??'),
('Frutas', 'Uva sem semente 500 g', '', 14.9, '??'),
('Verduras', 'Alface crespa', '', 3.9, '??'),
('Verduras', 'Cenoura 1 kg', '', 5.5, '??'),
('Verduras', 'Brócolis', '', 6.9, '??'),
('Legumes', 'Batata 1 kg', '', 6.2, '??'),
('Legumes', 'Cebola 1 kg', '', 5.8, '??'),
('Legumes', 'Abóbora', '', 4.9, '??'),
('Ovos', 'Ovos caipiras (12 un)', '', 16.0, '??')
) as v(section, name, description, price, emoji)
cross join (select id from public.stores where name = 'Hortifrúti da Terra') as st union all
select st.id, v.section, v.name, v.description, v.price, v.emoji
from (values
('Pizzas', 'Margherita', 'Molho de tomate, mussarela e manjericão.', 42.0, '??'),  ('Pizzas', 'Calabresa', 'Calabresa
fatiada e cebola.', 46.0, '??'),  ('Pizzas', 'Quatro queijos', 'Mussarela, parmesão, gorgonzola e catupiry.', 52.0,
'??'),  ('Bebidas', 'Refrigerante 2 L', '', 12.0, '??'),
('Bebidas', 'Suco natural 500 ml', '', 9.0, '??'),
('Sobremesas', 'Brownie com sorvete', '', 18.0, '??'),
('Sobremesas', 'Pudim', '', 12.0, '??')
) as v(section, name, description, price, emoji)
cross join (select id from public.stores where name = 'Pizzaria Forno a Lenha') as st union all
select st.id, v.section, v.name, v.description, v.price, v.emoji
from (values
('Combos', 'Combo 20 peças', '', 79.9, '??'),
('Combos', 'Combo 30 peças', '', 109.9, '??'),
('Individuais', 'Temaki de salmão', '', 24.9, '??'),
('Individuais', 'Hot roll (8 un)', '', 28.9, '??'),
('Bebidas', 'Chá gelado', '', 8.0, '??')
) as v(section, name, description, price, emoji)
cross join (select id from public.stores where name = 'Sushi Sakura') as st;
```


3.4 Facilitar o cadastro em sala

Para testar sem depender de e-mail, desative a confirmação: em **Authentication → Sign In / Providers → Email**, desligue **Confirm email**. O nome do menu pode variar levemente entre versões do painel. Em um app real, mantenha a confirmação ligada.

3.5 Copiar as chaves para o app

1. Em **Project Settings** → **API Keys** (ou **Data API**), copie a **Project URL**.
2. Copie a chave pública: a `anon` (legada) ou a `publishable`.
3. Cole os dois valores no arquivo `.env` do projeto:

.env Variáveis

```
EXPO_PUBLIC_SUPABASE_URL=https://abcdefghij.supabase.co
EXPO_PUBLIC_SUPABASE_KEY=eyJhbGciOi...sua-chave-publica
```

ATENÇÃO

Nunca copie a chave `service_role` nem a senha do banco para o app. Depois de alterar o `.env`, reinicie o servidor com `npx expo start -c`.

PONTO DE VERIFICAÇÃO

- Em **Table Editor**, `stores` tem 6 linhas e `products` tem 50. `orders` e `order_items` estão vazias, todas com RLS ativo.
- Em **Database** → **Functions**, existem `create_order`, `advance_order` e `cancel_order`.
- O `.env` contém a URL e a chave reais do seu projeto.

ATENÇÃO

Se o app disser que não encontra a função depois de você mudar o `schema.sql`, rode `notify pgrst, 'reload schema';` no SQL Editor para atualizar o cache da API.

Etapa 4 — Testar o banco no seu computador

Regras de segurança escritas em SQL precisam de testes, como qualquer código. O problema:

testar contra o Supabase real é lento e bagunça os dados. A solução é o **PGlite**, um Postgres de verdade compilado para rodar dentro do Node. Ele executa o **mesmo** `schema.sql` e `seed.sql`, sem Docker e sem internet.

4.1 O arquivo de testes

Crie a pasta `supabase/tests` e o arquivo abaixo:

Código 10 · supabase/tests/db.test.mjs Texto

```
// Testes do banco com PGlite (Postgres real, rodando dentro do Node, sem Docker). // Aplicamos o
MESMO schema.sql e seed.sql que você roda no Supabase.
// Execução: node --test supabase/tests/ (ou npm run test:db)
import assert from 'node:assert/strict';
import { readFileSync } from 'node:fs';
import { after, before, describe, it } from 'node:test';
import { fileURLToPath } from 'node:url';
import { PGlite } from '@electric-sql/pglite';

const SUPABASE = fileURLToPath(new URL('..', import.meta.url));
const ALICE = '11111111-1111-1111-1111-111111111111';
const BOB = '22222222-2222-2222-2222-222222222222';
const M = 'Mercado Bom Preço';

// O que o Supabase já oferece e o Postgres puro não tem: o schema "auth", a função // auth.uid() (lê
o usuário do "token") e os papéis anon e authenticated.
const SUPABASE_STUBS = `
 create schema auth;
 create table auth.users (id uuid primary key);
 create function auth.uid() returns uuid language sql stable as $$
 select nullif(current_setting('request.jwt.claim.sub', true), '')::uuid
 $$;
 create role anon nologin;
 create role authenticated nologin;
 grant usage on schema public, auth to anon, authenticated;
 insert into auth.users (id) values ('${ALICE}'), ('${BOB}');
`;

let db;

before(async () => {
 db = new PGlite();
 await db.exec(SUPABASE_STUBS);
 await db.exec(readFileSync(`${SUPABASE}/schema.sql`, 'utf8'));
 await db.exec(readFileSync(`${SUPABASE}/seed.sql`, 'utf8'));
});

after(async () => {
 await db.close();
});

// Executa como um usuário logado (uid) ou como visitante logado sem sessão (uid = null). async function
as(uid, fn) {
 await db.exec(
 `set role authenticated;
 select set_config('request.jwt.claim.sub', '${uid ?? ''}', false);`,
 );
 try {
 return await fn();
 } finally {
 await db.exec(`reset role; select set_config('request.jwt.claim.sub', '', false);`);  }
```



```
}
```

```
const one = async (sql, params = []) => (await db.query(sql, params)).rows[0]; const rows =
async (sql, params = []) => (await db.query(sql, params)).rows; const storeId = async (name) =>
 (await one('select id from stores where name = $1', [name])).id; const productId =
async (name, store) =>
 (
 await one(
 `select p.id from products p join stores s on s.id = p.store_id  where p.name = $1 and
s.name = $2`,
 [name, store],
 )
 ).id;

async function createOrder(uid, store, items, o = {}) {
 const sid = await storeId(store);
 return as(uid, async () => {
 const r = await db.query(
 'select public.create_order($1, $2::jsonb, $3, $4, $5, $6, $7, $8) as id',  [
 sid,
 JSON.stringify(items),
 o.payment ?? 'pix',
 o.changeFor ?? null,
 o.street ?? 'Rua das Flores',
 o.number ?? '120',
 o.neighborhood ?? 'Centro',
 o.complement ?? null,
 ],
 );
 return r.rows[0].id;
 });
}

const advance = (uid, id) =>
 as(uid, async () => (await one('select public.advance_order($1) as s', [id])).s);

describe('leitura e permissões', () => {
 it('usuário logado lê lojas e só os produtos ativos', async () => {  assert.equal((await as(ALICE, () =>
rows('select id from stores'))).length, 6);  const pid = await productId('Detergente 500 ml', M);
 await db.query('update products set is_active = false where id = $1', [pid]);  assert.equal(
 (await as(ALICE, () => rows('select id from products where id = $1', [pid])))  .length,
 0,
 );
 await db.query('update products set is_active = true where id = $1', [pid]);  });

 it('o app não consegue inserir pedidos diretamente', async () => {  const sid = await
storeId(M);
 await assert.rejects(
 as(ALICE, () =>
 db.query(
 `insert into orders (store_id, store_name, payment_method,  address_street,
address_number, address_neighborhood,  subtotal, delivery_fee, total)
 values ($1, 'x', 'pix', 'r', '1', 'b', 1, 0, 1)`,
 [sid],
 ),
 ),
 /permission denied/,
 );
 });

 it('visitante (anon) não executa as funções', async () => {
 await db.exec('set role anon');
```



```
 try {
 await assert.rejects(
 db.query('select public.advance_order(gen_random_uuid())'),
 /permission denied/,
```

```
  );
  } finally {
  await db.exec('reset role');
  }
  });
});

describe('create_order', () => {
  it('calcula o total no servidor e guarda os itens (snapshot)', async () => {  const leite = await
productId('Leite integral 1 L', M);
  const arroz = await productId('Arroz branco 5 kg', M);
  const id = await createOrder(ALICE, M, [
  { product_id: leite, quantity: 2 },
  { product_id: arroz, quantity: 1 },
  ]);

  const order = await one('select * from orders where id = $1', [id]);
assert.equal(order.status, 'received');
  assert.equal(order.subtotal, '40.40');
  assert.equal(order.delivery_fee, '5.90');
  assert.equal(order.total, '46.30');
  assert.equal(order.store_name, M);
  assert.equal(order.user_id, ALICE);

  const items = await rows(
  'select name, quantity, total_price from order_items where order_id = $1 order by name',  [id],
  );
  assert.deepEqual(items, [
  { name: 'Arroz branco 5 kg', quantity: 1, total_price: '29.90' },  { name: 'Leite integral 1 L',
quantity: 2, total_price: '10.50' },  ]);

  // o preço muda depois, mas o pedido antigo continua igual
  await db.query('update products set price = 99 where id = $1', [leite]);  assert.equal(
  (
  await one(
  'select unit_price from order_items where order_id = $1 and product_id = $2',  [id, leite],
  )
  ).unit_price,
  '5.25',
  );
  await db.query('update products set price = 5.25 where id = $1', [leite]);  });

  it('respeita o pedido mínimo da loja', async () => {
  const pao = await productId('Pão francês (6 un)', 'Padaria Pão Quente');  await assert.rejects(
  createOrder(ALICE, 'Padaria Pão Quente', [{ product_id: pao, quantity: 1 }]),  /O pedido mínimo desta
loja é R\$ 10,00\./,
  );
  });

  it('recusa loja fechada', async () => {
  const combo = await productId('Combo 20 peças', 'Sushi Sakura');
  await assert.rejects(
  createOrder(ALICE, 'Sushi Sakura', [{ product_id: combo, quantity: 1 }]),  /A loja está fechada no
momento\./,
  );
  });

  it('recusa itens inválidos: outra loja, inativo, quantidade e repetidos', async () => {  const leite = await
productId('Leite integral 1 L', M);
```



```
  const pizza = await productId('Margherita', 'Pizzaria Forno a Lenha');  const invalid = /Há
itens inválidos no carrinho/;

  await assert.rejects(
  createOrder(ALICE, M, [{ product_id: pizza, quantity: 1 }]),
  invalid,
```

```
);
await assert.rejects(
createOrder(ALICE, M, [{ product_id: leite, quantity: 0 }]),
invalid,
);
await assert.rejects(
createOrder(ALICE, M, [{ product_id: leite, quantity: 21 }]),
invalid,
);
await assert.rejects(
createOrder(ALICE, M, [
{ product_id: leite, quantity: 1 },
{ product_id: leite, quantity: 2 },
]),
invalid,
);

await db.query('update products set is_active = false where id = $1', [leite]);  await assert.rejects(
createOrder(ALICE, M, [{ product_id: leite, quantity: 5 }]),
invalid,
);
await db.query('update products set is_active = true where id = $1', [leite]);  });

it('valida carrinho vazio, login, endereço e pagamento', async () => {  const arroz = await
productId('Arroz branco 5 kg', M);
const item = [{ product_id: arroz, quantity: 1 }];
await assert.rejects(createOrder(ALICE, M, []), /O carrinho está vazio\./);  await assert.rejects(createOrder(null,
M, item), /Faça login para fazer um pedido\./);  await assert.rejects(
createOrder(ALICE, M, item, { street: ' ' }),
/Endereço de entrega incompleto\./,
);
await assert.rejects(
createOrder(ALICE, M, item, { payment: 'boleto' }),
/Forma de pagamento inválida\./,
);
});

it('troco: só vale em dinheiro e precisa cobrir o total', async () => {  const arroz = await
productId('Arroz branco 5 kg', M); // 29,90 + 5,90 = 35,80  const item = [{ product_id: arroz, quantity: 1
}];
await assert.rejects(
createOrder(ALICE, M, item, { payment: 'cash', changeFor: 30 }),  /troco/,
);
 const cash = await createOrder(ALICE, M, item, { payment: 'cash', changeFor: 50 });  const card = await
createOrder(ALICE, M, item, { payment: 'card', changeFor: 50 });  assert.equal(
(await one('select change_for from orders where id = $1', [cash])).change_for,  '50.00',
);
assert.equal(
(await one('select change_for from orders where id = $1', [card])).change_for,  null,
);
});
});

describe('privacidade e status', () => {
it('cada usuário só vê os próprios pedidos e itens', async () => {  const arroz = await
productId('Arroz branco 5 kg', M);
const id = await createOrder(BOB, M, [{ product_id: arroz, quantity: 1 }]);
```



```
assert.equal(
(await as(ALICE, () => rows('select id from orders where id = $1', [id]))).length,  0,
);
assert.equal(
(await as(BOB, () => rows('select id from orders where id = $1', [id]))).length,  1,
);
assert.equal(
(
await as(ALICE, () =>
```

```
rows('select id from order_items where order_id = $1', [id]),
)
).length,
0,
);
assert.ok(
(await as(BOB, () => rows('select id from order_items where order_id = $1', [id]))) .length > 0,
);
});

it('o pedido avança na ordem certa e para em entregue', async () => {
const arroz = await productId('Arroz branco 5 kg', M);
const id = await createOrder(ALICE, M, [{ product_id: arroz, quantity: 1 }]); assert.equal(await
advance(ALICE, id), 'preparing');
assert.equal(await advance(ALICE, id), 'out_for_delivery');
assert.equal(await advance(ALICE, id), 'delivered');
await assert.rejects(advance(ALICE, id), /Este pedido não pode avançar\./); });

it('outro usuário não avança nem cancela o pedido alheio', async () => {
const arroz = await productId('Arroz branco 5 kg', M);
const id = await createOrder(ALICE, M, [{ product_id: arroz, quantity: 1 }]); await
assert.rejects(advance(BOB, id), /Pedido não encontrado\./);
await assert.rejects(
as(BOB, () => db.query('select public.cancel_order($1)', [id])),
/Pedido não encontrado\./,
);
});

it('só cancela enquanto o pedido está recebido', async () => {
const arroz = await productId('Arroz branco 5 kg', M);
const a = await createOrder(ALICE, M, [{ product_id: arroz, quantity: 1 }]); await as(ALICE, () =>
db.query('select public.cancel_order($1)', [a]));
assert.equal(
(await one('select status from orders where id = $1', [a])).status,
'cancelled',
);
await assert.rejects(advance(ALICE, a), /Este pedido não pode avançar\./);

const b = await createOrder(ALICE, M, [{ product_id: arroz, quantity: 1 }]); await advance(ALICE,
b);
await assert.rejects(
as(ALICE, () => db.query('select public.cancel_order($1)', [b])),
/cancelar/,
);
});
});
```

Como ele funciona

- `SUPABASE_STUBS` recria o que o Supabase oferece e o Postgres puro não tem: o schema `auth`, a função `auth.uid()` e os papéis `anon` e `authenticated`.
- `as(usuário, função)` executa um trecho **como aquele usuário**: troca de papel com `set role authenticated` e informa quem está logado. É assim que o RLS é testado de verdade.

- Os testes cobrem: leitura permitida, **escrita direta recusada**, cálculo do total no servidor, snapshot de preço, pedido mínimo, loja fechada, itens inválidos (outra loja, inativo, quantidade fora da faixa, repetido), carrinho vazio, login, endereço, pagamento, troco, privacidade entre usuários, ordem dos status e cancelamento.
- O arquivo é `.mjs` (JavaScript moderno) e usa o executor nativo do Node (`node --test`), sem depender do Jest.

4.2 Executar

Terminal bash
```
npm run test:db
```

Saída esperada (resumida) Texto
```
✔ leitura e permissões
✔ create_order
✔ privacidade e status

# tests 13
# pass 13
# fail 0
```

PONTO DE VERIFICAÇÃO

- O comando termina em poucos segundos com **13 testes passando** e `# fail 0`.
- Você não precisou de internet, de Docker nem de tocar no banco do Supabase.

DICA

Quebre de propósito. No `schema.sql`, apague a linha `where i.quantity between 1 and 20` da função `create_order` e rode `npm run test:db`. Veja qual teste acusa a quantidade inválida. Depois desfaça. Repare que o teste pegou uma falha de **segurança** sem abrir o app.

BOM SABER

Os testes rodam sobre o arquivo `schema.sql` do disco, e não sobre o banco do Supabase. Se você alterar o banco pelo painel sem atualizar o arquivo, os testes deixam de refletir a realidade. Por isso, **toda mudança no banco entra primeiro no arquivo**.

Etapa 5 — Fundações: tipos, tema e conexão

Antes de qualquer tela, criamos a base compartilhada: os tipos do domínio, as constantes do catálogo, o tema visual, a conexão com o backend e utilitários de apoio.

5.1 Criar a estrutura de pastas

No terminal (no Windows, use o Git Bash, que vem com o Git):

Terminal bash

```
mkdir -p "app/(auth)" "app/(app)/(tabs)" "app/(app)/store" "app/(app)/product" "app/(app)/order" mkdir -p
src/components src/constants src/context src/hooks src/lib
mkdir -p src/schemas src/services src/theme src/types src/utils
mkdir -p __tests__ supabase/tests
```

BOM SABER

Prefere criar pelo VS Code? Use o botão **New Folder** do Explorer. Os nomes precisam ficar idênticos, inclusive os parênteses das pastas em `app/`.

5.2 Tipos do domínio

Código 11 · src/types/index.ts TypeScript

```
export type StoreCategory =
'Mercado' | 'Padaria' | 'Farmácia' | 'Restaurante' | 'Hortifrúti';

export interface Store {
 id: string;
 name: string;
 category: StoreCategory;
 emoji: string;
 description: string;
 rating: number;
 delivery_fee: number;
 eta_min: number;
 eta_max: number;
 min_order: number;
 is_open: boolean;
}

export interface Product {
 id: string;
 store_id: string;
 section: string;
 name: string;
 description: string;
 price: number;
 emoji: string;
 is_active: boolean;
}

export type PaymentMethod = 'pix' | 'card' | 'cash';

export type OrderStatus =
'received' | 'preparing' | 'out_for_delivery' | 'delivered' | 'cancelled';
```



```
export interface OrderItem {
 id: string;
 order_id: string;
 product_id: string | null;
 name: string;
 emoji: string;
 unit_price: number;
 quantity: number;
```

```
  total_price: number;
}

export interface Order {
  id: string;
  store_id: string;
  store_name: string;
  status: OrderStatus;
  payment_method: PaymentMethod;
  change_for: number | null;
  address_street: string;
  address_number: string;
  address_neighborhood: string;
  address_complement: string | null;
  subtotal: number;
  delivery_fee: number;
  total: number;
  created_at: string;
  updated_at: string;
  items?: OrderItem[];
}

// ---- carrinho (guardado no aparelho) ----
export interface CartItem {
  productId: string;
  name: string;
  emoji: string;
  price: number;
  quantity: number;
}

export interface Cart {
  storeId: string | null;
  storeName: string;
  deliveryFee: number;
  minOrder: number;
  items: CartItem[];
}

// O que o app envia ao criar um pedido. Repare: NÃO há preços aqui.
export interface CreateOrderInput {
  storeId: string;
  items: { product_id: string; quantity: number }[];
  payment: PaymentMethod;
  changeFor: number | null;
  street: string;
  number: string;
  neighborhood: string;
  complement: string;
}
```

- Os tipos espelham as tabelas do banco (com os nomes das colunas em `snake_case`) e o carrinho, que vive só no aparelho (em `camelCase`).
- `CreateOrderInput` é o que o app **envia** ao criar um pedido. Repare que **não existe nenhum campo de preço**: só ids e quantidades.
- `Order` e `OrderItem` guardam os dados **copiados** na hora da compra.

5.3 Constantes do catálogo

Código 12 · src/constants/catalog.ts TypeScript `import type { OrderStatus, PaymentMethod, StoreCategory } from`

```
'@/types';

export const STORE_CATEGORIES: { name: StoreCategory; emoji: string }[] = [
 { name: 'Mercado', emoji: '' },
 { name: 'Padaria', emoji: '' },
 { name: 'Farmácia', emoji: '' },
 { name: 'Hortifrúti', emoji: '' },
 { name: 'Restaurante', emoji: '' },
];

export const PAYMENT_METHODS: { value: PaymentMethod; label: string; hint: string }[] = [  { value: 'pix',
label: 'Pix', hint: 'Pagamento simulado' },
 { value: 'card', label: 'Cartão na entrega', hint: 'Levamos a maquininha' },  { value: 'cash', label:
'Dinheiro', hint: 'Informe o troco, se precisar' },
];

// Ordem em que o pedido avança. "cancelled" fica fora da linha do tempo.
export const ORDER_FLOW: OrderStatus[] = [
 'received',
 'preparing',
 'out_for_delivery',
 'delivered',
];

export const ORDER_STATUS_LABEL: Record<OrderStatus, string> = {
 received: 'Pedido recebido',
 preparing: 'Em preparo',
 out_for_delivery: 'Saiu para entrega',
 delivered: 'Entregue',
 cancelled: 'Cancelado',
};

export const ORDER_STATUS_COLOR: Record<OrderStatus, { fg: string; bg: string }> = {  received: {
fg: '#B45309', bg: '#FEF3C7' },
 preparing: { fg: '#1D4ED8', bg: '#DBEAFE' },
 out_for_delivery: { fg: '#7C3AED', bg: '#EDE9FE' },
 delivered: { fg: '#15803D', bg: '#DCFCE7' },
 cancelled: { fg: '#B91C1C', bg: '#FEE2E2' },
};
```

Categorias de loja, formas de pagamento, a ordem dos status e os textos e cores de cada status ficam em um lugar só. `ORDER_FLOW` define a sequência da linha do tempo, e `cancelled` fica de fora dela.

5.4 Tema (design tokens)

Código 13 · src/theme/index.ts TypeScript

```ts
export const colors = {
 primary: '#EA580C',
 primarySoft: '#FFEDD5',
 background: '#F4F6F8',
 surface: '#FFFFFF',
 text: '#0F172A',
 muted: '#64748B',
 border: '#E2E8F0',
 income: '#16A34A',
 expense: '#DC2626',
 danger: '#DC2626',
};

export const spacing = { xs: 4, sm: 8, md: 16, lg: 24, xl: 32 };

export const radius = { sm: 8, md: 12, lg: 20, pill: 999 };

export const shadow = {
 shadowColor: '#0F172A',
 shadowOpacity: 0.08,
 shadowRadius: 8,
 shadowOffset: { width: 0, height: 2 },
 elevation: 2,
};
```

Cores, espaçamentos e raios ficam em um único lugar. A cor primária deste projeto é o laranja.

5.5 Cliente do Supabase e do React Query

Código 14 · src/lib/supabase.ts TypeScript

```ts
import 'react-native-url-polyfill/auto';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { createClient } from '@supabase/supabase-js';
import { AppState } from 'react-native';

const url = process.env.EXPO_PUBLIC_SUPABASE_URL;
const key = process.env.EXPO_PUBLIC_SUPABASE_KEY;

if (!url || !key) {
 throw new Error(
 'Variáveis do Supabase ausentes. Confira o .env e reinicie com: npx expo start -c',  );
}

export const supabase = createClient(url, key, {
 auth: {
 storage: AsyncStorage,
 autoRefreshToken: true,
 persistSession: true,
 detectSessionInUrl: false,
 },
});

// Renova o token só enquanto o app está em primeiro plano.
AppState.addEventListener('change', (state) => {
 if (state === 'active') supabase.auth.startAutoRefresh();
 else supabase.auth.stopAutoRefresh();
});
```

• O import de `react-native-url-polyfill/auto` precisa vir **primeiro**.

• `AsyncStorage` guarda a sessão no aparelho, por isso o usuário continua logado ao reabrir o app. • O `AppState` renova o token apenas com o app em primeiro plano.

Código 15 · src/lib/queryClient.ts TypeScript

```ts
import { QueryClient } from '@tanstack/react-query';

export const queryClient = new QueryClient({
 defaultOptions: {
 queries: { retry: 1, staleTime: 30_000 },
 },
});
```

5.6 Utilitários: erros, ícones, dinheiro e data

Código 16 · src/utils/errors.ts TypeScript

```ts
// Traduz mensagens comuns do Supabase e da rede para português.
export function friendlyError(error: unknown): string {
 const message =
 typeof error === 'object' && error !== null && 'message' in error
 ? String((error as { message: unknown }).message)
 : String(error);

 if (message.includes('Invalid login credentials')) {
 return 'E-mail ou senha incorretos.';
 }
 if (message.includes('User already registered')) {
 return 'Este e-mail já está cadastrado.';
 }
 if (message.includes('Email not confirmed')) {
 return 'Confirme seu e-mail antes de entrar.';
 }
 if (/network request failed|failed to fetch/i.test(message)) {
 return 'Sem conexão com a internet.';
 }
 if (/rate limit/i.test(message)) {
 return 'Muitas tentativas. Aguarde um instante e tente de novo.';
 }
 return message;
}
```

As mensagens das funções do banco (“O pedido mínimo desta loja é R$ 20,00.”) já chegam em português e passam direto. Aqui traduzimos apenas as do Supabase e da rede.

Código 17 · src/utils/icons.ts TypeScript

```ts
import { Ionicons } from '@expo/vector-icons';
import type { ComponentProps } from 'react';

export type IconName = ComponentProps<typeof Ionicons>['name'];

// Garante um ícone válido mesmo se o banco tiver um nome desconhecido.
export function safeIcon(name: string): IconName {
 return name in Ionicons.glyphMap ? (name as IconName) : 'pricetag';
}
```

Código 18 · src/utils/money.ts TypeScript

```ts
const brl = new Intl.NumberFormat('pt-BR', {
 style: 'currency',
 currency: 'BRL',
});

export function formatMoney(value: number): string {
 return brl.format(value);
}

// Converte texto digitado pelo usuário ("1.234,56", "10,5", "12.5") em número. // Retorna NaN
quando o texto não representa um valor.
export function parseMoney(text: string): number {
 const clean = text.replace(/[^\d.,]/g, '');
 if (!/\d/.test(clean)) return NaN;

 let normalized = clean;
 if (clean.includes(',')) {
 normalized = clean.replace(/\./g, '').replace(',', '.');
 } else if (/^\d{1,3}(\.\d{3})+$/.test(clean)) {
 normalized = clean.replace(/\./g, '');
 }
 return Number(normalized);
}

export function roundMoney(value: number): number {
 return Math.round(value * 100) / 100;
}

// Valor numérico -> texto de campo ("1234.5" -> "1234,50").
export function toInputMoney(value: number): string {
 return value.toFixed(2).replace('.', ',');
}
```

- `formatMoney` gera “R$ 1.234,50” com `Intl.NumberFormat`.
- `parseMoney` entende o que o usuário digita: `1.234,56`, `10,5`, `12.5` e `1.234` (milhar). Será usado no campo de troco.

Código 19 · src/utils/date.ts TypeScript `const pad = (n: number) => String(n).padStart(2, '0');`

```ts
// "2026-09-19T14:30:00Z" -> "19/09 às 11:30" (horário local do aparelho).
export function formatDateTime(isoDateTime: string): string {
 const date = new Date(isoDateTime);
 if (Number.isNaN(date.getTime())) return '';
 const day = `${pad(date.getDate())}/${pad(date.getMonth() + 1)}`;
 return `${day} às ${pad(date.getHours())}:${pad(date.getMinutes())}`;
}
```

Terminal bash `npm run typecheck`

PONTO DE VERIFICAÇÃO

- O comando termina **sem nenhuma mensagem de erro**.
- Se aparecer erro sobre `process`, confira se o arquivo `expo-env.d.ts` existe (criado por `npx expo start`).

Etapa 6 — Regras de negócio e testes automatizados

A lógica do carrinho, do catálogo e dos pedidos fica em **funções puras**: sem tela, sem banco e sem rede. Assim, testamos em segundos, direto no terminal. É a parte do projeto que mais rende em entrevistas, porque mostra que você separa **regra** de **interface**.

6.1 O carrinho

Código 20 · src/utils/cart.ts TypeScript

```
import type { Cart, CartItem, Order, Product, Store } from '@/types';

export const MAX_QUANTITY = 20;

export const EMPTY_CART: Cart = {
 storeId: null,
 storeName: '',
 deliveryFee: 0,
 minOrder: 0,
 items: [],
};

type CartStore = Pick<Store, 'id' | 'name' | 'delivery_fee' | 'min_order'>;
type CartProduct = Pick<Product, 'id' | 'name' | 'emoji' | 'price'>;

 const clamp = (n: number) => Math.max(0, Math.min(MAX_QUANTITY, Math.floor(n))); const
 cents = (value: number) => Math.round(value * 100);

function newCart(store: CartStore): Cart {
 return {
 storeId: store.id,
 storeName: store.name,
 deliveryFee: store.delivery_fee,
 minOrder: store.min_order,
 items: [],
 };
}

function withItem(cart: Cart, product: CartProduct, quantity: number): Cart {
 const existing = cart.items.find((i) => i.productId === product.id);
 const total = clamp((existing?.quantity ?? 0) + quantity);
 if (total === 0)
 return { ...cart, items: cart.items.filter((i) => i.productId !== product.id) };  const item: CartItem = {
 productId: product.id,
 name: product.name,
 emoji: product.emoji,
 price: product.price,
 quantity: total,
 };
 return existing
 ? { ...cart, items: cart.items.map((i) => (i.productId === product.id ? item : i)) }  : { ...cart, items: [...cart.items,
item] };
}

// Regra do delivery: o carrinho é de UMA loja por vez. Se o produto for de outra loja, // devolvemos
"conflict" e a tela pergunta se o usuário quer trocar.
export function addToCart(
 cart: Cart,
 store: CartStore,
 product: CartProduct,
 quantity = 1,
): { cart: Cart; conflict: boolean } {
 if (cart.storeId && cart.storeId !== store.id && cart.items.length > 0) {
```

```
 return { cart, conflict: true };
 }
 const base = cart.storeId === store.id ? cart : newCart(store);
 return { cart: withItem(base, product, quantity), conflict: false }; }

// Começa um carrinho novo (descarta o anterior).
export function replaceCart(store: CartStore, product: CartProduct, quantity = 1): Cart {  return
withItem(newCart(store), product, quantity);
}

export function setQuantity(cart: Cart, productId: string, quantity: number): Cart {  const q =
clamp(quantity);
 const items =
 q === 0
 ? cart.items.filter((i) => i.productId !== productId)
 : cart.items.map((i) => (i.productId === productId ? { ...i, quantity: q } : i));  return items.length === 0 ?
EMPTY_CART : { ...cart, items };
}

export function removeItem(cart: Cart, productId: string): Cart {
 return setQuantity(cart, productId, 0);
}

export function itemCount(cart: Cart): number {
 return cart.items.reduce((sum, i) => sum + i.quantity, 0);
}

export function quantityOf(cart: Cart, productId: string): number {  return cart.items.find((i)
=> i.productId === productId)?.quantity ?? 0; }

// Soma em centavos inteiros para não acumular erro de ponto flutuante. export function
cartTotals(cart: Cart): {
 subtotal: number;
 deliveryFee: number;
 total: number;
} {
 const sub = cart.items.reduce((sum, i) => sum + cents(i.price) * i.quantity, 0);  const fee =
cart.items.length > 0 ? cents(cart.deliveryFee) : 0;
 return { subtotal: sub / 100, deliveryFee: fee / 100, total: (sub + fee) / 100 }; }

// Quanto falta para atingir o pedido mínimo da loja (null se já atingiu). export function
missingForMinimum(cart: Cart): number | null {
 const missing = cents(cart.minOrder) - cents(cartTotals(cart).subtotal);  return missing > 0 ?
missing / 100 : null;
}

// O carrinho vem do AsyncStorage: confira o formato antes de confiar nele. export function
isValidCart(value: unknown): value is Cart {
 if (typeof value !== 'object' || value === null) return false;
 const c = value as Record<string, unknown>;
 if (!(c.storeId === null || typeof c.storeId === 'string')) return false;  if (
 typeof c.storeName !== 'string' ||
 typeof c.deliveryFee !== 'number' ||
 typeof c.minOrder !== 'number'
 ) {
 return false;
 }
 if (!Array.isArray(c.items)) return false;
 return c.items.every((i) => {
 if (typeof i !== 'object' || i === null) return false;
 const item = i as Record<string, unknown>;
 return (
 typeof item.productId === 'string' &&
 typeof item.name === 'string' &&
 typeof item.emoji === 'string' &&
```

```
 typeof item.price === 'number' &&
 typeof item.quantity === 'number' &&
 item.quantity >= 1 &&
 item.quantity <= MAX_QUANTITY
 );
 });
}

// "Pedir novamente": monta o carrinho com os itens de um pedido antigo, usando os // preços e a
disponibilidade ATUAIS. Produtos que saíram do catálogo ficam de fora. export function reorderCart(
 order: Pick<Order, 'items'>,
 store: CartStore,
 products: Pick<Product, 'id' | 'name' | 'emoji' | 'price' | 'is_active'>[],
): { cart: Cart; skipped: string[] } {
 let cart = newCart(store);
 const skipped: string[] = [];
 for (const item of order.items ?? []) {
 const product = products.find((p) => p.id === item.product_id && p.is_active);  if (!product)
skipped.push(item.name);
 else cart = withItem(cart, product, item.quantity);
 }
 return { cart: cart.items.length > 0 ? cart : EMPTY_CART, skipped };
}
```

- **Uma loja por vez.** `addToCart` devolve `conflict: true` se o produto for de outra loja, **sem alterar o carrinho**. Quem decide o que fazer é a tela.
- **Limite de 20 por produto** (`MAX_QUANTITY`), o mesmo do banco. A regra existe nos dois lados: no app para a experiência, no banco para a segurança.
- **Soma em centavos.** Em JavaScript, `0.1 + 0.2` vale `0.30000000000000004`. `cartTotals` soma inteiros e divide por 100 no fim.
- **«missingForMinimum»** diz quanto falta para o pedido mínimo, ou `null` se já atingiu.
- **«isValidCart»**: o carrinho vem do armazenamento do aparelho e pode estar corrompido ou ser de uma versão antiga. Conferimos o formato antes de confiar.
- **«reorderCart»** (“Pedir novamente”): monta um carrinho com os itens de um pedido antigo, mas com os **preços atuais**, deixando de fora o que saiu do catálogo.

6.2 Catálogo e pedidos

Código 21 · src/utils/catalog.ts TypeScript

```
import type { Product, Store, StoreCategory } from '@/types';

const normalize = (s: string) =>
 s
 .toLowerCase()
 .normalize('NFD')
 .replace(/[\u0300-\u036f]/g, '');

// Busca por nome, categoria ou descrição (sem acento nem maiúsculas) e filtro por categoria. export function
filterStores(
 stores: Store[],
 query: string,
 category: StoreCategory | null,
): Store[] {
 const q = normalize(query.trim());
 return stores.filter((s) => {
```

```
 if (category && s.category !== category) return false;
 return !q || normalize(`${s.name} ${s.category} ${s.description}`).includes(q);  });
}

// Lojas abertas primeiro; dentro de cada grupo, melhor avaliação primeiro.
export function sortStores(stores: Store[]): Store[] {
 return [...stores].sort(
 (a, b) => Number(b.is_open) - Number(a.is_open) || b.rating - a.rating,
 );
}

export interface ProductSection {
 title: string;
 data: Product[];
}

// Agrupa por seção mantendo a ordem em que cada seção aparece pela primeira vez. export
function groupProducts(products: Product[]): ProductSection[] {
 const sections = new Map<string, Product[]>();
 for (const p of products) {
 const list = sections.get(p.section);
 if (list) list.push(p);
 else sections.set(p.section, [p]);
 }
 return Array.from(sections, ([title, data]) => ({ title, data }));
}

export const formatEta = (min: number, max: number) => `${min}–${max} min`;
```

- `filterStores` busca por nome, categoria e descrição, sem acento e sem diferença de maiúsculas (`normalize('NFD')`).
- `sortStores` coloca as lojas abertas primeiro e ordena pela avaliação, sem alterar a lista original. • `groupProducts` agrupa por seção mantendo a ordem em que cada seção aparece.

Código 22 · src/utils/orders.ts TypeScript

```ts
import { ORDER_FLOW } from '@/constants/catalog';
import type { OrderStatus } from '@/types';

export type StepState = 'done' | 'current' | 'pending';

// Linha do tempo do pedido: cada etapa está concluída, em andamento ou pendente. export
function timeline(
 status: OrderStatus,
): { status: OrderStatus; state: StepState }[] {
 const current = ORDER_FLOW.indexOf(status);
 return ORDER_FLOW.map((s, i) => ({
 status: s,
 state:
 status === 'cancelled'
 ? 'pending'
 : i < current || status === 'delivered'
 ? 'done'
 : i === current
 ? 'current'
 : 'pending',
 }));
}

export function nextStatus(status: OrderStatus): OrderStatus | null {
 const i = ORDER_FLOW.indexOf(status);
 return i >= 0 && i < ORDER_FLOW.length - 1 ? ORDER_FLOW[i + 1] : null;
}

// Enquanto o pedido está em andamento, a tela consulta o servidor de tempos em tempos. export const
isActive = (status: OrderStatus) =>
 status !== 'delivered' && status !== 'cancelled';

export const canCancel = (status: OrderStatus) => status === 'received';
```

- `timeline` diz o estado de cada etapa: concluída, atual ou pendente. Um pedido cancelado não avança nenhuma etapa.
- `isActive` decide se a tela deve continuar atualizando o pedido.

6.3 Validação do checkout

Código 23 · src/schemas/checkout.ts TypeScript

```
import { z } from 'zod';
import { parseMoney } from '@/utils/money';

// O total muda a cada carrinho, então o schema é criado por uma função.
export function makeCheckoutSchema(total: number) {
 return z
 .object({
 street: z.string().trim().min(3, 'Informe a rua'),
 number: z
 .string()
 .trim()
 .min(1, 'Informe o número')
 .max(10, 'Máximo de 10 caracteres'),
 neighborhood: z.string().trim().min(2, 'Informe o bairro'),
 complement: z.string().trim().max(60, 'Máximo de 60 caracteres'),
 payment: z.enum(['pix', 'card', 'cash']),
 changeFor: z.string(),
 })
 .superRefine((data, ctx) => {
 if (data.payment !== 'cash' || data.changeFor.trim() === '') return;
 const value = parseMoney(data.changeFor);
 if (Number.isNaN(value) || value <= 0) {
 ctx.addIssue({ code: 'custom', path: ['changeFor'], message: 'Valor inválido' });  } else if (value < total) {
 ctx.addIssue({
 code: 'custom',
 path: ['changeFor'],
 message: 'O troco deve cobrir o total do pedido',
 });
 }
```

```
});
}

export type CheckoutFormData = z.infer<ReturnType<typeof makeCheckoutSchema>>;
```

- O schema é criado por uma função, `makeCheckoutSchema(total)`, porque a regra do troco depende do total do carrinho.
- O `superRefine` só valida o troco quando o pagamento é em dinheiro: o troco precisa ser um valor válido e cobrir o total.

6.4 Testes automatizados

Código 24 · __tests__/money.test.ts TypeScript

```
import { formatMoney, parseMoney, roundMoney, toInputMoney }
from '@/utils/money';

const nbsp = /\u00a0/g;

describe('parseMoney', () => {
 it('aceita vírgula decimal', () => {
 expect(parseMoney('10,5')).toBe(10.5);
 expect(parseMoney('0,99')).toBe(0.99);
 });

 it('aceita ponto como separador de milhar', () => {
 expect(parseMoney('1.234,56')).toBe(1234.56);
 expect(parseMoney('1.234')).toBe(1234);
 });

 it('aceita ponto decimal', () => {
 expect(parseMoney('12.5')).toBe(12.5);
 });

 it('retorna NaN para texto sem números', () => {
 expect(parseMoney('')).toBeNaN();
 expect(parseMoney('abc')).toBeNaN();
 });
});

describe('formatMoney', () => {
 it('formata em reais', () => {
```

```
 expect(formatMoney(1234.5).replace(nbsp, ' ')).toBe('R$ 1.234,50');
 });
});

describe('roundMoney / toInputMoney', () => {
 it('arredonda para 2 casas', () => {
 expect(roundMoney(0.1 + 0.2)).toBe(0.3);
 });

 it('gera texto de campo com vírgula', () => {
 expect(toInputMoney(1234.5)).toBe('1234,50');
 });
});
```

Código 25 · __tests__/cart.test.ts TypeScript

```
import type { Order, Product, Store } from '@/types';
import {
 EMPTY_CART,
 MAX_QUANTITY,
 addToCart,
 cartTotals,
 isValidCart,
 itemCount,
 missingForMinimum,
 quantityOf,
 removeItem,
 reorderCart,
 replaceCart,
 setQuantity,
} from '@/utils/cart';

const mercado = { id: 's1', name: 'Mercado', delivery_fee: 5.9, min_order: 20 }; const padaria = { id:
's2', name: 'Padaria', delivery_fee: 3.5, min_order: 10 };
```



```
const leite = { id: 'p1', name: 'Leite', emoji: '🥛', price: 5.25 }; const arroz = { id: 'p2',
name: 'Arroz', emoji: '🍚', price: 29.9 }; const pao = { id: 'p3', name: 'Pão', emoji:
'🍞', price: 4.8 };

describe('addToCart', () => {
 it('cria o carrinho da loja e soma a quantidade do mesmo produto', () => { let { cart } =
addToCart(EMPTY_CART, mercado, leite, 2);
 ({ cart } = addToCart(cart, mercado, leite, 3));
 expect(cart.storeId).toBe('s1');
 expect(cart.items).toHaveLength(1);
 expect(quantityOf(cart, 'p1')).toBe(5);
 });

 it('avisa o conflito ao misturar lojas e não altera o carrinho', () => { const { cart } =
addToCart(EMPTY_CART, mercado, leite);
 const result = addToCart(cart, padaria, pao);
 expect(result.conflict).toBe(true);
 expect(result.cart).toBe(cart);
 });

 it('não permite passar do limite de 20 por produto', () => {
 const { cart } = addToCart(EMPTY_CART, mercado, leite, 18);
 expect(quantityOf(addToCart(cart, mercado, leite, 10).cart, 'p1')).toBe(MAX_QUANTITY); });

 it('replaceCart descarta o carrinho anterior', () => {
 const cart = replaceCart(padaria, pao, 2);
 expect(cart.storeId).toBe('s2');
 expect(itemCount(cart)).toBe(2);
 });
```

```
});

describe('setQuantity e removeItem', () => {
 it('altera a quantidade, limita em 20 e remove com zero', () => {
 let { cart } = addToCart(EMPTY_CART, mercado, leite, 2);
 ({ cart } = addToCart(cart, mercado, arroz, 1));
 expect(quantityOf(setQuantity(cart, 'p1', 7), 'p1')).toBe(7);
 expect(quantityOf(setQuantity(cart, 'p1', 99), 'p1')).toBe(20);
 expect(setQuantity(cart, 'p1', 0).items.map((i) => i.productId)).toEqual(['p2']);  });

 it('volta ao carrinho vazio quando o último item sai', () => {
 const { cart } = addToCart(EMPTY_CART, mercado, leite);
 expect(removeItem(cart, 'p1')).toBe(EMPTY_CART);
 });
});

describe('cartTotals e pedido mínimo', () => {
 it('soma em centavos e inclui a taxa de entrega', () => {
 let { cart } = addToCart(EMPTY_CART, mercado, leite, 2); // 10,50  ({ cart } =
addToCart(cart, mercado, arroz, 1)); // 29,90
 expect(cartTotals(cart)).toEqual({ subtotal: 40.4, deliveryFee: 5.9, total: 46.3 });  });

 it('não cobra entrega de carrinho vazio', () => {
 expect(cartTotals(EMPTY_CART)).toEqual({ subtotal: 0, deliveryFee: 0, total: 0 });  });

 it('evita o erro clássico 0,1 + 0,2', () => {
 const { cart } = addToCart(
 EMPTY_CART,
 { ...mercado, delivery_fee: 0 },
 { ...leite, price: 0.1 },
 1,
 );
 const both = addToCart(
 cart,
 { ...mercado, delivery_fee: 0 },
```



```
 { id: 'x', name: 'X', emoji: '??', price: 0.2 },
 1,
 ).cart;
 expect(cartTotals(both).subtotal).toBe(0.3);
 });

 it('diz quanto falta para o pedido mínimo', () => {
 const { cart } = addToCart(EMPTY_CART, mercado, leite, 2); // 10,50 de 20,00
expect(missingForMinimum(cart)).toBe(9.5);
 expect(missingForMinimum(addToCart(cart, mercado, arroz).cart)).toBeNull();  });
});

describe('isValidCart', () => {
 it('aceita um carrinho válido e recusa formatos quebrados', () => {
 const { cart } = addToCart(EMPTY_CART, mercado, leite);
 expect(isValidCart(cart)).toBe(true);
 expect(isValidCart(EMPTY_CART)).toBe(true);
 expect(isValidCart(null)).toBe(false);
 expect(isValidCart({ ...cart, items: [{ productId: 'p1' }] })).toBe(false);  expect(isValidCart({ ...cart,
deliveryFee: '5' })).toBe(false);
 expect(isValidCart({ ...cart, items: [{ ...cart.items[0], quantity: 0 }] })).toBe(  false,
 );
 });
});

describe('reorderCart', () => {
 const store = mercado as Store;
 const order = {
 items: [
 { product_id: 'p1', name: 'Leite', quantity: 2 },
```

```
{ product_id: 'p2', name: 'Arroz', quantity: 1 },
{ product_id: null, name: 'Produto antigo', quantity: 1 },
],
} as Order;
const products = [
{ ...leite, price: 5.9, is_active: true },
{ ...arroz, is_active: false },
] as Product[];

it('usa o preço atual e ignora produtos indisponíveis', () => {
const { cart, skipped } = reorderCart(order, store, products);
expect(cart.items).toHaveLength(1);
expect(cart.items[0]).toMatchObject({ productId: 'p1', price: 5.9, quantity: 2 });
expect(skipped).toEqual(['Arroz', 'Produto antigo']);
});

it('devolve carrinho vazio se nada está disponível', () => {
expect(reorderCart(order, store, []).cart).toBe(EMPTY_CART);
});
});
```

Código 26 · __tests__/catalog.test.ts TypeScript

```
import type { Product, Store } from '@/types';
import { filterStores, formatEta, groupProducts, sortStores } from '@/utils/catalog';

const store = (partial: Partial<Store> & Pick<Store, 'id' | 'name'>): Store => ({  category: 'Mercado',
emoji: '🛒',
description: '',
rating: 4.5,
delivery_fee: 5,
eta_min: 30,
eta_max: 45,
min_order: 0,
```



```
is_open: true,
...partial,
});

const stores = [
store({ id: '1', name: 'Mercado Bom Preço', rating: 4.7 }),
store({ id: '2', name: 'Padaria Pão Quente', category: 'Padaria', rating: 4.8 }),  store({
id: '3',
name: 'Farmácia Vida',
category: 'Farmácia',
rating: 4.6,
description: 'Medicamentos',
}),
store({
id: '4',
name: 'Sushi Sakura',
category: 'Restaurante',
rating: 4.9,
is_open: false,
}),
];

describe('filterStores', () => {
it('busca sem acento e sem diferença de maiúsculas', () => {
expect(filterStores(stores, 'FARMACIA', null).map((s) => s.id)).toEqual(['3']);
expect(filterStores(stores, 'pao', null).map((s) => s.id)).toEqual(['2']);  });

it('busca também na descrição', () => {
expect(filterStores(stores, 'medicamentos', null).map((s) => s.id)).toEqual(['3']);  });
```

```
 it('filtra por categoria e combina com a busca', () => {
 expect(filterStores(stores, '', 'Padaria').map((s) => s.id)).toEqual(['2']);  expect(filterStores(stores,
'bom', 'Padaria')).toEqual([]);
 });
});

describe('sortStores', () => {
 it('coloca as abertas primeiro e ordena pela avaliação', () => {
expect(sortStores(stores).map((s) => s.id)).toEqual(['2', '1', '3', '4']);  });

 it('não altera a lista original', () => {
 const copy = [...stores];
 sortStores(stores);
 expect(stores).toEqual(copy);
 });
});

describe('groupProducts', () => {
 const p = (id: string, section: string) => ({ id, section }) as Product;

 it('agrupa por seção mantendo a ordem da primeira aparição', () => {  const groups =
groupProducts([p('a', 'Pães'), p('b', 'Doces'), p('c', 'Pães')]);  expect(groups.map((g) =>
g.title)).toEqual(['Pães', 'Doces']);  expect(groups[0].data.map((x) => x.id)).toEqual(['a', 'c']);
 });
});

it('formatEta usa travessão', () => {
 expect(formatEta(30, 45)).toBe('30–45 min');
});
```


Código 27 · __tests__/orders.test.ts TypeScript

```
import { canCancel, isActive, nextStatus, timeline } from
'@/utils/orders';

describe('timeline', () => {
 it('marca concluídas, a atual e as pendentes', () => {
 expect(timeline('preparing').map((s) => s.state)).toEqual([
 'done',
 'current',
 'pending',
 'pending',
 ]);
 });

 it('entregue deixa todas as etapas concluídas', () => {
 expect(timeline('delivered').every((s) => s.state === 'done')).toBe(true);  });

 it('cancelado não avança nenhuma etapa', () => {
 expect(timeline('cancelled').every((s) => s.state === 'pending')).toBe(true);  });
});

describe('status', () => {
 it('nextStatus segue a ordem e termina em entregue', () => {
 expect(nextStatus('received')).toBe('preparing');
 expect(nextStatus('out_for_delivery')).toBe('delivered');
 expect(nextStatus('delivered')).toBeNull();
 expect(nextStatus('cancelled')).toBeNull();
 });

 it('só está ativo o que não terminou', () => {
 expect(isActive('preparing')).toBe(true);
```

```
 expect(isActive('delivered')).toBe(false);
 expect(isActive('cancelled')).toBe(false);
 });

 it('só cancela um pedido recém-recebido', () => {
 expect(canCancel('received')).toBe(true);
 expect(canCancel('preparing')).toBe(false);
 });
});
```

Código 28 · __tests__/checkout.test.ts TypeScript

```
import { makeCheckoutSchema } from '@/schemas/checkout';

const valid = {
 street: 'Rua das Flores',
 number: '120',
 neighborhood: 'Centro',
 complement: '',
 payment: 'pix' as const,
 changeFor: '',
};

describe('makeCheckoutSchema', () => {
 const schema = makeCheckoutSchema(46.3);
 const errorsOf = (data: unknown) => {
 const r = schema.safeParse(data);
 return r.success
 ? []
 : r.error.issues.map((i) => `${String(i.path[0])}: ${i.message}`); };

 it('aceita um pedido completo', () => {
 expect(schema.safeParse(valid).success).toBe(true);
```



```
 });

 it('exige rua, número e bairro', () => {
 expect(errorsOf({ ...valid, street: 'ab', number: '', neighborhood: ' ' })).toEqual([ 'street: Informe a rua',
 'number: Informe o número',
 'neighborhood: Informe o bairro',
 ]);
 });

 it('troco: só é validado em dinheiro', () => {
 expect(schema.safeParse({ ...valid, payment: 'pix', changeFor: '10' }).success).toBe( true,
 );
 expect(errorsOf({ ...valid, payment: 'cash', changeFor: '30' })).toEqual([ 'changeFor: O troco
deve cobrir o total do pedido',
 ]);
 expect(errorsOf({ ...valid, payment: 'cash', changeFor: 'abc' })).toEqual([ 'changeFor: Valor
inválido',
 ]);
 expect(
 schema.safeParse({ ...valid, payment: 'cash', changeFor: '50,00' }).success, ).toBe(true);
 expect(schema.safeParse({ ...valid, payment: 'cash', changeFor: '' }).success).toBe( true,
 );
 });

 it('recusa forma de pagamento desconhecida', () => {
 expect(schema.safeParse({ ...valid, payment: 'boleto' }).success).toBe(false); });
});
```

Terminal bash

```
npm run test:app
```

Saída esperada Texto

```
PASS __tests__/cart.test.ts
PASS __tests__/catalog.test.ts
PASS __tests__/orders.test.ts
PASS __tests__/checkout.test.ts
PASS __tests__/money.test.ts

Test Suites: 5 passed, 5 total
Tests: 37 passed, 37 total
```

PONTO DE VERIFICAÇÃO

- `npm run test:app` mostra 5 suítes e **37 testes passando**.
- `npm run typecheck` continua sem erros.
- Repare nos testes de **conflito de loja**, de **limite de 20** e de **carrinho corrompido**: são regras que só se descobrem testando.

DICA

Quebre de propósito. Em `cart.ts`, apague o bloco `if (cart.storeId && ...)` dentro de `addToCart` (a verificação de conflito) e rode `npm run test:app`. Veja quais testes falham. Depois desfaça.

Etapa 7 — Camada de dados: serviços e hooks

Construímos o caminho **tela** → **hook** → **serviço** → **Supabase**, sem escrever nenhuma tela ainda.

7.1 Validação do login e do cadastro

Código 29 · src/schemas/auth.ts TypeScript

```ts
import { z } from 'zod';

const email = z.string().trim().min(1, 'Informe o e-mail').email('E-mail inválido');

export const loginSchema = z.object({
 email,
 password: z.string().min(1, 'Informe a senha'),
});

export const registerSchema = z
 .object({
 name: z.string().trim().min(2, 'Informe seu nome'),
 email,
 password: z.string().min(6, 'Use pelo menos 6 caracteres'),
 confirm: z.string().min(1, 'Confirme a senha'),
 })
 .refine((data) => data.password === data.confirm, {
 message: 'As senhas não conferem',
 path: ['confirm'],
 });

export type LoginData = z.infer<typeof loginSchema>;
export type RegisterData = z.infer<typeof registerSchema>;
```

7.2 Serviços

Código 30 · src/services/stores.ts TypeScript

```
import { supabase } from '@/lib/supabase';
import type { Product, Store } from '@/types';

export async function listStores(): Promise<Store[]> {
 const { data, error } = await supabase.from('stores').select('*').order('name');  if (error) throw error;
 return (data ?? []) as Store[];
}

export async function getStore(id: string): Promise<Store> {
 const { data, error } = await supabase.from('stores').select('*').eq('id', id).single();  if (error) throw error;
 return data as Store;
}

export async function listProducts(storeId: string): Promise<Product[]> {  const { data, error
} = await supabase
 .from('products')
 .select('*')
 .eq('store_id', storeId)
 .order('section')
 .order('name');
 if (error) throw error;
 return (data ?? []) as Product[];
}

export async function getProduct(id: string): Promise<Product> {
 const { data, error } = await supabase
 .from('products')
 .select('*')
 .eq('id', id)
```

```
.single();
 if (error) throw error;
 return data as Product;
}
```

Código 31 · src/services/orders.ts TypeScript

```
import { supabase } from '@/lib/supabase';
import type { CreateOrderInput, Order } from '@/types';

// A ÚNICA forma de criar um pedido: a função create_order, no banco. Ela confere os // itens, calcula o
total com os preços do banco e devolve o id do pedido. export async function createOrder(input:
CreateOrderInput): Promise<string> {  const { data, error } = await supabase.rpc('create_order', {
 p_store_id: input.storeId,
 p_items: input.items,
 p_payment: input.payment,
 p_change_for: input.changeFor,
 p_street: input.street,
 p_number: input.number,
 p_neighborhood: input.neighborhood,
 p_complement: input.complement,
 });
 if (error) throw error;
 return data as string;
}

export async function listOrders(): Promise<Order[]> {
 const { data, error } = await supabase
 .from('orders')
 .select('*')
 .order('created_at', { ascending: false });
```



```
 if (error) throw error;
 return (data ?? []) as Order[];
}

export async function getOrder(id: string): Promise<Order> {
 const { data, error } = await supabase
 .from('orders')
 .select('*, items:order_items(*)')
 .eq('id', id)
 .single();
 if (error) throw error;
 return data as Order;
}

// Modo demonstração: em um produto real, quem muda o status é a loja.
export async function advanceOrder(id: string): Promise<void> {
 const { error } = await supabase.rpc('advance_order', { p_order_id: id });
 if (error) throw error;
}

export async function cancelOrder(id: string): Promise<void> {
 const { error } = await supabase.rpc('cancel_order', { p_order_id: id });
 if (error) throw error;
}

// Endereço do último pedido, para preencher o checkout.
export async function getLastAddress(): Promise<Pick<
 Order,
 'address_street' | 'address_number' | 'address_neighborhood' | 'address_complement' > | null> {
 const { data, error } = await supabase
 .from('orders')
 .select('address_street, address_number, address_neighborhood, address_complement')
.order('created_at', { ascending: false })
 .limit(1)
```

```
.maybeSingle();
if (error) throw error;
return data;
}
```

- **«createOrder» é a estrela.** Chama a função do banco com `supabase.rpc('create_order', ...)` enviando **só ids, quantidades, pagamento e endereço**. Devolve o id do pedido.
- `listOrders` e `getOrder` apenas **leem**. O RLS garante que só os pedidos do usuário voltam, sem nenhum filtro no código.
- `getOrder` usa `select('*, items:order_items(*)')` para trazer o pedido já com os itens, em uma consulta.
- `getLastAddress` usa `maybeSingle`: se o usuário ainda não fez pedidos, devolve `null` em vez de erro. • `advanceOrder` e `cancelOrder` também são chamadas de função (RPC).

7.3 Hooks com TanStack Query

Código 32 · src/hooks/useCatalog.ts TypeScript

```
import { useQuery } from '@tanstack/react-query';
import { getProduct, getStore, listProducts, listStores } from '@/services/stores';

export const useStores = () =>
 useQuery({ queryKey: ['stores'], queryFn: listStores, staleTime: 60_000 });

export function useStore(id: string | undefined) {
 return useQuery({
 queryKey: ['stores', id],
 queryFn: () => getStore(id as string),
 enabled: Boolean(id),
 staleTime: 60_000,
 });
}

export function useProducts(storeId: string | undefined) {
 return useQuery({
 queryKey: ['products', 'store', storeId],
 queryFn: () => listProducts(storeId as string),
 enabled: Boolean(storeId),
 staleTime: 60_000,
 });
}

export function useProduct(id: string | undefined) {
 return useQuery({
 queryKey: ['products', 'detail', id],
 queryFn: () => getProduct(id as string),
```

```
  enabled: Boolean(id),
  staleTime: 60_000,
 });
}
```

Código 33 · src/hooks/useOrders.ts TypeScript

```
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query'; import {
 advanceOrder,
 cancelOrder,
 createOrder,
 getLastAddress,
 getOrder,
 listOrders,
} from '@/services/orders';
import { isActive } from '@/utils/orders';

export const useOrders = () =>
 useQuery({ queryKey: ['orders', 'list'], queryFn: listOrders });

// Enquanto o pedido está em andamento, consulta de novo a cada 5 segundos. export
function useOrder(id: string | undefined) {
 return useQuery({
  queryKey: ['orders', 'detail', id],
  queryFn: () => getOrder(id as string),
  enabled: Boolean(id),
  refetchInterval: (query) => {
   const status = query.state.data?.status;
   return status && isActive(status) ? 5000 : false;
  },
 });
}
```



```
export const useLastAddress = () =>
 useQuery({
  queryKey: ['orders', 'last-address'],
  queryFn: getLastAddress,
  staleTime: Infinity,
 });

function useInvalidateOrders() {
 const queryClient = useQueryClient();
 return () => queryClient.invalidateQueries({ queryKey: ['orders'] });
}

export function useCreateOrder() {
 const invalidate = useInvalidateOrders();
 return useMutation({ mutationFn: createOrder, onSuccess: invalidate });
}

export function useAdvanceOrder() {
 const invalidate = useInvalidateOrders();
 return useMutation({
  mutationFn: (id: string) => advanceOrder(id),
  onSuccess: invalidate,
 });
}

export function useCancelOrder() {
 const invalidate = useInvalidateOrders();
 return useMutation({
  mutationFn: (id: string) => cancelOrder(id),
  onSuccess: invalidate,
 });
```

```
}
```

Código 34 · src/hooks/useReorder.ts TypeScript

```ts
import { useMutation } from '@tanstack/react-query';
import { getStore, listProducts } from '@/services/stores';
import type { Order } from '@/types';
import { reorderCart } from '@/utils/cart';

// "Pedir novamente": busca os preços e a disponibilidade atuais e monta o carrinho. export function
useReorder() {
 return useMutation({
 mutationFn: async (order: Order) => {
 const [store, products] = await Promise.all([
 getStore(order.store_id),
 listProducts(order.store_id),
 ]);
 return reorderCart(order, store, products);
 },
 });
}
```

- O catálogo muda pouco: `staleTime` de 60 segundos evita consultas repetidas ao navegar entre telas.
- **«useOrder» atualiza sozinho.** O `refetchInterval` recebe uma função: enquanto o pedido está em andamento, consulta de novo a cada 5 segundos; quando termina (`entregue` ou `cancelado`), para. É o “acompanhe em tempo real” sem complicação. No Apêndice C, há o desafio de trocar isso por Realtime.
- Toda mutação invalida a chave `['orders']`: lista e detalhe atualizam juntos.
- `useReorder` busca a loja e os produtos atuais e usa `reorderCart`. Os dados chegam à tela pronto, e a tela nunca fala com o serviço.

Terminal bash `npm run typecheck`

PONTO DE VERIFICAÇÃO

- Nenhum erro de tipo.

Etapa 8 — Design system: componentes de interface

Componentes pequenos e reutilizáveis dão identidade ao app. Os seis primeiros são a base (os mesmos do FinTrack); os seis últimos são específicos do MarketGo.

8.1 Base

Código 35 · src/components/Button.tsx TSX

```tsx
import { ActivityIndicator, Pressable, StyleSheet, Text } from 'react-native'; import type {
StyleProp, ViewStyle } from 'react-native';
import { colors, radius, spacing } from '@/theme';

type Variant = 'primary' | 'secondary' | 'danger' | 'ghost';

interface ButtonProps {
 title: string;
 onPress: () => void;
 variant?: Variant;
 loading?: boolean;
 disabled?: boolean;
 style?: StyleProp<ViewStyle>;
}

const labelColor: Record<Variant, string> = {
 primary: '#FFFFFF',
 danger: '#FFFFFF',
 secondary: colors.primary,
 ghost: colors.muted,
};

export function Button({
 title,
 onPress,
 variant = 'primary',
 loading = false,
 disabled = false,
 style,
}: ButtonProps) {
 const inactive = disabled || loading;
 return (
 <Pressable
 accessibilityRole="button"
 accessibilityState={{ disabled: inactive }}
 onPress={onPress}
 disabled={inactive}
 style={({ pressed }) => [
 styles.base,
 variantStyles[variant],
 pressed && styles.pressed,
 inactive && styles.inactive,
 style,
 ]}
 >
 {loading ? (
 <ActivityIndicator color={labelColor[variant]} />
 ) : (
 <Text style={[styles.label, { color: labelColor[variant] }]}>{title}</Text>  )}
 </Pressable>
 );
}

const styles = StyleSheet.create({
 base: {
```



```tsx
 minHeight: 52,
 borderRadius: radius.md,
 paddingHorizontal: spacing.md,
 alignItems: 'center',
```

```
 justifyContent: 'center',
 },
 label: { fontSize: 16, fontWeight: '700' },
 pressed: { opacity: 0.85 },
 inactive: { opacity: 0.5 },
});

const variantStyles = StyleSheet.create({
 primary: { backgroundColor: colors.primary },
 secondary: { backgroundColor: colors.primarySoft },
 danger: { backgroundColor: colors.danger },
 ghost: { backgroundColor: 'transparent' },
});
```

Código 36 · src/components/Input.tsx TSX

```
import { StyleSheet, Text, TextInput, View } from 'react-native';
import type { TextInputProps } from 'react-native';
import { colors, radius, spacing } from '@/theme';

interface InputProps extends TextInputProps {
 label: string;
 error?: string;
}

export function Input({ label, error, style, ...rest }: InputProps) {  return (
 <View style={styles.wrapper}>
 <Text style={styles.label}>{label}</Text>
 <TextInput
 placeholderTextColor={colors.muted}
 style={[styles.input, error ? styles.inputError : null, style]}  {...rest}
 />
 {error ? <Text style={styles.error}>{error}</Text> : null}
 </View>
 );
}

const styles = StyleSheet.create({
 wrapper: { marginBottom: spacing.md },
 label: { fontSize: 14, fontWeight: '600', color: colors.text, marginBottom: 6 },  input: {
 minHeight: 52,
 borderWidth: 1,
 borderColor: colors.border,
 borderRadius: radius.md,
 backgroundColor: colors.surface,
 paddingHorizontal: spacing.md,
 fontSize: 16,
 color: colors.text,
 },
 inputError: { borderColor: colors.danger },
 error: { color: colors.danger, fontSize: 13, marginTop: 4 },
});
```


Código 37 · src/components/FormInput.tsx TSX

```
import { Controller } from 'react-hook-form';
import type { Control, FieldValues, Path } from 'react-hook-form';
```

```tsx
import type { TextInputProps } from 'react-native';
import { Input } from './Input';

type FormInputProps<T extends FieldValues> = Omit<
 TextInputProps,
 'value' | 'onChangeText' | 'onBlur'
> & {
 control: Control<T>;
 name: Path<T>;
 label: string;
 mask?: (text: string) => string;
};

// Liga o <Input> ao react-hook-form e exibe a mensagem de erro do Zod. export
function FormInput<T extends FieldValues>({
 control,
 name,
 label,
 mask,
 ...rest
}: FormInputProps<T>) {
 return (
 <Controller
 control={control}
 name={name}
 render={({ field: { value, onChange, onBlur }, fieldState: { error } }) => (  <Input
 {...rest}
 label={label}
 value={String(value ?? '')}
 onChangeText={(text) => onChange(mask ? mask(text) : text)}  onBlur={onBlur}
 error={error?.message}
 />
 )}
 />
 );
}
```

`FormInput` liga o campo ao React Hook Form e exibe o erro do Zod sob o campo.

Código 38 · src/components/Card.tsx TSX

```tsx
import { StyleSheet, View } from 'react-native';
import type { StyleProp, ViewStyle } from 'react-native';
```

```
import type { ReactNode } from 'react';
import { colors, radius, shadow, spacing } from '@/theme';

interface CardProps {
 children: ReactNode;
 style?: StyleProp<ViewStyle>;
}

export function Card({ children, style }: CardProps) {
 return <View style={[styles.card, style]}>{children}</View>;
}

const styles = StyleSheet.create({
 card: {
 backgroundColor: colors.surface,
 borderRadius: radius.lg,
 padding: spacing.md,
 marginBottom: spacing.md,
 ...shadow,
 },
});
```

Código 39 · src/components/Screen.tsx TSX

```
import { KeyboardAvoidingView, Platform, ScrollView, StyleSheet } from 'react-native'; import type {
ReactNode } from 'react';
import { SafeAreaView } from 'react-native-safe-area-context';
import type { Edge } from 'react-native-safe-area-context';
import { colors, spacing } from '@/theme';

interface ScreenProps {
 children: ReactNode;
 scroll?: boolean;
 edges?: Edge[];
}

export function Screen({
 children,
 scroll = false,
 edges = ['top', 'left', 'right'],
}: ScreenProps) {
 return (
 <SafeAreaView style={styles.safe} edges={edges}>
 {scroll ? (
 <KeyboardAvoidingView
 style={styles.flex}
 behavior={Platform.OS === 'ios' ? 'padding' : undefined}  >
 <ScrollView
 contentContainerStyle={styles.content}
 keyboardShouldPersistTaps="handled"
 >
 {children}
 </ScrollView>
 </KeyboardAvoidingView>
 ) : (
 children
 )}
 </SafeAreaView>
 );
}

const styles = StyleSheet.create({
 safe: { flex: 1, backgroundColor: colors.background },
```



```
 flex: { flex: 1 },
 content: { padding: spacing.md, flexGrow: 1 },
```

```
});
```

Código 40 · src/components/StateViews.tsx TSX

```tsx
import { Ionicons } from '@expo/vector-icons';
import { ActivityIndicator, StyleSheet, Text, View } from 'react-native';
import { colors, spacing } from '@/theme';
import type { IconName } from '@/utils/icons';
import { Button } from './Button';

export function LoadingView() {
 return (
 <View style={styles.center}>
 <ActivityIndicator size="large" color={colors.primary} />
 </View>
 );
}

interface ErrorStateProps {
 message: string;
 onRetry: () => void;
}

export function ErrorState({ message, onRetry }: ErrorStateProps) {
 return (
 <View style={styles.center}>
 <Ionicons name="cloud-offline" size={48} color={colors.muted} />
 <Text style={styles.title}>Algo deu errado</Text>
 <Text style={styles.text}>{message}</Text>
 <Button title="Tentar novamente" onPress={onRetry} style={styles.button} />  </View>
 );
}

interface EmptyStateProps {
 icon?: IconName;
 title: string;
 message: string;
}

export function EmptyState({ icon = 'wallet-outline', title, message }: EmptyStateProps) {  return (
 <View style={styles.center}>
 <Ionicons name={icon} size={48} color={colors.muted} />
 <Text style={styles.title}>{title}</Text>
 <Text style={styles.text}>{message}</Text>
 </View>
 );
}

 const styles = StyleSheet.create({
 center: {
 flex: 1,
 alignItems: 'center',
 justifyContent: 'center',
 padding: spacing.lg,
 gap: spacing.sm,
 },
 title: { fontSize: 18, fontWeight: '700', color: colors.text },
 text: { fontSize: 14, color: colors.muted, textAlign: 'center' },
 button: { marginTop: spacing.sm, alignSelf: 'stretch' },
 });
```

Toda tela com dados remotos precisa tratar **carregando**, **erro** (com botão para tentar de novo) e **vazio**.

8.2 Componentes do MarketGo

Código 41 · src/components/QuantityStepper.tsx TSX

```tsx
import { Ionicons } from '@expo/vector-icons';
import { Pressable, StyleSheet, Text, View } from 'react-native';
import { colors, radius } from '@/theme';
import { MAX_QUANTITY } from '@/utils/cart';

interface QuantityStepperProps {
 value: number;
 onChange: (next: number) => void;
 min?: number;
 max?: number;
}

export function QuantityStepper({
 value,
 onChange,
 min = 0,
 max = MAX_QUANTITY,
}: QuantityStepperProps) {
 return (
 <View style={styles.row}>
 <Pressable
 accessibilityRole="button"
 accessibilityLabel="Diminuir quantidade"
 disabled={value <= min}
 onPress={() => onChange(value - 1)}
 style={[styles.button, value <= min && styles.disabled]}
 >
 <Ionicons
 name={value === 1 && min === 0 ? 'trash-outline' : 'remove'}  size={20}
 color={colors.primary}
 />
 </Pressable>
 <Text style={styles.value}>{value}</Text>
 <Pressable
 accessibilityRole="button"
 accessibilityLabel="Aumentar quantidade"
 disabled={value >= max}
 onPress={() => onChange(value + 1)}
 style={[styles.button, value >= max && styles.disabled]}
 >
 <Ionicons name="add" size={20} color={colors.primary} />
 </Pressable>
 </View>
 );
}

const styles = StyleSheet.create({
 row: { flexDirection: 'row', alignItems: 'center', gap: 4 },
 button: {
 width: 36,
 height: 36,
 borderRadius: radius.pill,
 backgroundColor: colors.primarySoft,
 alignItems: 'center',
 justifyContent: 'center',
 },
 disabled: { opacity: 0.4 },
 value: {
 minWidth: 28,
 textAlign: 'center',
 fontSize: 16,
 fontWeight: '700',
 color: colors.text,
 },
});
```

- Botões de menos e mais com limites (`min` e `max`). Quando a quantidade é 1 e o mínimo é 0, o ícone vira uma **lixeira**: tocar remove o item.
- Todo botão tem `accessibilityRole` e `accessibilityLabel`: leitores de tela e testes automatizados encontram o elemento pelo nome.

Código 42 · src/components/CategoryChips.tsx TSX

```
import { Pressable, ScrollView, StyleSheet, Text } from 'react-native';
import { STORE_CATEGORIES } from '@/constants/catalog';
import { colors, radius, spacing } from '@/theme';
import type { StoreCategory } from '@/types';

interface CategoryChipsProps {
 value: StoreCategory | null;
 onChange: (next: StoreCategory | null) => void;
}

// Toque em uma categoria para filtrar; toque de novo para limpar.
export function CategoryChips({ value, onChange }: CategoryChipsProps) {
 return (
 <ScrollView
 horizontal
 showsHorizontalScrollIndicator={false}
 contentContainerStyle={styles.row}
 >
 {STORE_CATEGORIES.map((c) => {
 const selected = c.name === value;
 return (
 <Pressable
 key={c.name}
 onPress={() => onChange(selected ? null : c.name)}
 style={[styles.chip, selected && styles.chipSelected]}
 >
 <Text style={styles.emoji}>{c.emoji}</Text>
 <Text style={[styles.label, selected && styles.labelSelected]}>{c.name}</Text>  </Pressable>
 );
 })}
 </ScrollView>
 );
}

const styles = StyleSheet.create({
 row: { gap: spacing.sm, paddingVertical: spacing.sm },
 chip: {
 flexDirection: 'row',
 alignItems: 'center',
 gap: 6,
 backgroundColor: colors.surface,
 borderWidth: 1,
 borderColor: colors.border,
 borderRadius: radius.pill,
 paddingVertical: 8,
 paddingHorizontal: 14,
 },
 chipSelected: { backgroundColor: colors.primary, borderColor: colors.primary },  emoji: { fontSize:
16 },
 label: { fontSize: 14, fontWeight: '600', color: colors.text },
 labelSelected: { color: '#FFFFFF' },
});
```

Filtro horizontal por categoria. Tocar na categoria ativa **limpa** o filtro.

Código 43 · src/components/StoreCard.tsx TSX

```tsx
import { Pressable, StyleSheet, Text, View } from 'react-native';
import { colors, radius, shadow, spacing } from '@/theme';
import type { Store } from '@/types';
import { formatEta } from '@/utils/catalog';
import { formatMoney } from '@/utils/money';

interface StoreCardProps {
 store: Store;
 onPress: () => void;
}

export function StoreCard({ store, onPress }: StoreCardProps) {
 return (
 <Pressable
 accessibilityRole="button"
 onPress={onPress}
 style={({ pressed }) => [
 styles.card,
 pressed && styles.pressed,
 !store.is_open && styles.closed,
 ]}
 >
 <View style={styles.logo}>
 <Text style={styles.emoji}>{store.emoji}</Text>
 </View>
 <View style={styles.info}>
 <Text style={styles.name} numberOfLines={1}>
 {store.name}
 </Text>
 <Text style={styles.meta}>
 {store.category} · ⭐ {store.rating.toFixed(1).replace('.', ',')}  </Text>
 {store.is_open ? (
 <Text style={styles.meta}>
 {formatEta(store.eta_min, store.eta_max)} ·{' '}
 {store.delivery_fee === 0
 ? 'Entrega grátis'
 : formatMoney(store.delivery_fee)}
 </Text>
 ) : (
 <View style={styles.badge}>
 <Text style={styles.badgeText}>Fechada agora</Text>
 </View>
 )}
 </View>
 </Pressable>
 );
}

const styles = StyleSheet.create({
 card: {
 flexDirection: 'row',
 alignItems: 'center',
 gap: spacing.md,
 backgroundColor: colors.surface,
 borderRadius: radius.lg,
 padding: spacing.md,
 marginBottom: spacing.sm,
 ...shadow,
 },
 pressed: { opacity: 0.85 },
```

```
closed: { opacity: 0.6 },
logo: {
width: 64,
height: 64,
borderRadius: 20,
backgroundColor: colors.primarySoft,
alignItems: 'center',
justifyContent: 'center',
```



```
},
emoji: { fontSize: 32 },
info: { flex: 1, gap: 2 },
name: { fontSize: 17, fontWeight: '700', color: colors.text },
meta: { fontSize: 13, color: colors.muted },
badge: {
alignSelf: 'flex-start',
backgroundColor: '#FEE2E2',
borderRadius: radius.pill,
paddingHorizontal: 10,
paddingVertical: 3,
marginTop: 2,
},
badgeText: { fontSize: 12, fontWeight: '700', color: colors.danger },
});
```

Cartão da loja: emoji, nome, categoria, avaliação, tempo e taxa de entrega. Lojas fechadas ficam esmaecidas e mostram “Fechada agora”. Usamos **emojis** como imagens, para o projeto não depender de arquivos de imagem.

Código 44 · src/components/ProductRow.tsx TSX

```tsx
import { Ionicons } from '@expo/vector-icons';
import { Pressable, StyleSheet, Text, View } from 'react-native';
import { colors, radius, spacing } from '@/theme';
import type { Product } from '@/types';
import { formatMoney } from '@/utils/money';

interface ProductRowProps {
product: Product;
quantityInCart: number;
disabled?: boolean;
onPress: () => void;
onQuickAdd: () => void;
}

export function ProductRow({
product,
quantityInCart,
disabled,
onPress,
onQuickAdd,
}: ProductRowProps) {
return (
<Pressable
accessibilityRole="button"
onPress={onPress}
style={({ pressed }) => [styles.row, pressed && styles.pressed]}
>
<View style={styles.thumb}>
<Text style={styles.emoji}>{product.emoji}</Text>
</View>
<View style={styles.info}>
<Text style={styles.name} numberOfLines={2}>
{product.name}
</Text>
```

```
{product.description ? (
<Text style={styles.description} numberOfLines={2}>
{product.description}
</Text>
) : null}
<Text style={styles.price}>{formatMoney(product.price)}</Text>
</View>
<Pressable
accessibilityRole="button"
accessibilityLabel={`Adicionar ${product.name}`}
disabled={disabled}
```



```
onPress={onQuickAdd}
style={[styles.add, disabled && styles.addDisabled]}
>
{quantityInCart > 0 ? (
<Text style={styles.addCount}>{quantityInCart}</Text>
) : (
<Ionicons name="add" size={24} color="#FFFFFF" />
)}
</Pressable>
</Pressable>
);
}
```

```
const styles = StyleSheet.create({
row: {
flexDirection: 'row',
alignItems: 'center',
gap: spacing.md,
backgroundColor: colors.surface,
borderRadius: radius.md,
padding: spacing.md,
marginBottom: spacing.sm,
},
pressed: { opacity: 0.85 },
thumb: {
width: 56,
height: 56,
borderRadius: 16,
backgroundColor: colors.background,
alignItems: 'center',
justifyContent: 'center',
},
emoji: { fontSize: 30 },
info: { flex: 1, gap: 2 },
name: { fontSize: 16, fontWeight: '600', color: colors.text },
description: { fontSize: 13, color: colors.muted },
price: { fontSize: 15, fontWeight: '800', color: colors.text, marginTop: 2 },  add: {
width: 40,
height: 40,
borderRadius: 20,
backgroundColor: colors.primary,
alignItems: 'center',
justifyContent: 'center',
},
addDisabled: { backgroundColor: colors.border },
addCount: { color: '#FFFFFF', fontWeight: '800', fontSize: 16 },
});
```

Linha do produto. O botão redondo mostra “+” ou, se o item já está no carrinho, a **quantidade**. O toque no restante da linha abre o detalhe.

Código 45 · src/components/StatusPill.tsx TSX

```
import { StyleSheet, Text, View } from 'react-native';
import { ORDER_STATUS_COLOR, ORDER_STATUS_LABEL } from '@/constants/catalog';
import { radius } from '@/theme';
import type { OrderStatus } from '@/types';

export function StatusPill({ status }: { status: OrderStatus }) {
 const { fg, bg } = ORDER_STATUS_COLOR[status];
 return (
 <View style={[styles.pill, { backgroundColor: bg }]}>
 <Text style={[styles.text, { color: fg }]}>{ORDER_STATUS_LABEL[status]}</Text>  </View>
 );
}

const styles = StyleSheet.create({
 pill: {
 alignSelf: 'flex-start',
 borderRadius: radius.pill,
 paddingHorizontal: 10,
 paddingVertical: 3,
 },
 text: { fontSize: 12, fontWeight: '700' },
});
```

Código 46 · src/components/OrderTimeline.tsx TSX

```
import { Ionicons } from '@expo/vector-icons';
import { StyleSheet, Text, View } from 'react-native';
import { ORDER_STATUS_LABEL } from '@/constants/catalog';
import { colors, spacing } from '@/theme';
import type { OrderStatus } from '@/types';
import { timeline } from '@/utils/orders';

// Linha do tempo vertical: concluída (verde), atual (laranja) e pendente (cinza). export function
OrderTimeline({ status }: { status: OrderStatus }) {  const steps = timeline(status);
 return (
 <View>
 {steps.map((step, index) => {
 const done = step.state === 'done';
 const current = step.state === 'current';
 const color = done ? colors.income : current ? colors.primary : colors.border;  return (
 <View key={step.status} style={styles.row}>
 <View style={styles.rail}>
 <View style={[styles.dot, { backgroundColor: color }]}>  {done ? <Ionicons name="checkmark" size={14}
color="#FFFFFF" /> : null}  </View>
 {index < steps.length - 1 ? (
 <View style={[styles.line, done && { backgroundColor: colors.income }]} />  ) : null}
 </View>
```

```
<Text
style={[
styles.label,
current && styles.current,
step.state === 'pending' && styles.pending,
]}
>
{ORDER_STATUS_LABEL[step.status]}
</Text>
</View>
);
})}
</View>
);
}
```



```
const styles = StyleSheet.create({
row: { flexDirection: 'row', gap: spacing.md, minHeight: 48 },
rail: { alignItems: 'center', width: 24 },
dot: {
width: 24,
height: 24,
borderRadius: 12,
alignItems: 'center',
justifyContent: 'center',
},
line: { flex: 1, width: 3, backgroundColor: colors.border, marginVertical: 2 },  label: { fontSize: 16,
color: colors.text, paddingTop: 1 },
current: { fontWeight: '800', color: colors.primary },
pending: { color: colors.muted },
});
```

Linha do tempo vertical: concluída (verde, com marca de confirmação), atual (laranja) e pendente (cinza). O estado de cada etapa vem da função pura `timeline`, testada na Etapa 6.

```
Terminal bash npm run typecheck
```

PONTO DE VERIFICAÇÃO

- Nenhum erro de tipo. Os componentes ganham vida nas próximas etapas.

Etapa 9 — Sessão e carrinho: os contextos

O app precisa de dois estados globais: **quem é o usuário** e **o que há no carrinho**. Os dois vivem em contextos do React.

9.1 Sessão do usuário

Código 47 · src/context/AuthContext.tsx TSX

```
import type { Session, User } from '@supabase/supabase-js';
import { createContext, useContext, useEffect, useMemo, useState } from 'react'; import type {
ReactNode } from 'react';
import { queryClient } from '@/lib/queryClient';
import { supabase } from '@/lib/supabase';

interface AuthContextValue {
 session: Session | null;
 user: User | null;
 loading: boolean;
 signIn: (email: string, password: string) => Promise<void>;
 signUp: (
 name: string,
 email: string,
 password: string,
 ) => Promise<{ needsConfirmation: boolean }>;
 signOut: () => Promise<void>;
```

```
}

const AuthContext = createContext<AuthContextValue | undefined>(undefined);

export function AuthProvider({ children }: { children: ReactNode }) {
 const [session, setSession] = useState<Session | null>(null);
 const [loading, setLoading] = useState(true);

 useEffect(() => {
 supabase.auth.getSession().then(({ data }) => {
 setSession(data.session);
 setLoading(false);
 });

 const { data } = supabase.auth.onAuthStateChange((_event, newSession) => {
setSession(newSession);
 if (!newSession) queryClient.clear(); // não vazar dados entre contas
 });

 return () => data.subscription.unsubscribe();
 }, []);

 const value = useMemo<AuthContextValue>(
 () => ({
 session,
 user: session?.user ?? null,
 loading,
 signIn: async (email, password) => {
 const { error } = await supabase.auth.signInWithPassword({ email, password });  if (error) throw error;
 },
 signUp: async (name, email, password) => {
 const { data, error } = await supabase.auth.signUp({
 email,
 password,
 options: { data: { full_name: name } },
 });
 if (error) throw error;
```



```
 return { needsConfirmation: data.session === null };
 },
 signOut: async () => {
 const { error } = await supabase.auth.signOut();
 if (error) throw error;
 },
 }),
 [session, loading],
 );

 return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>; }

export function useAuth(): AuthContextValue {
 const context = useContext(AuthContext);
 if (!context) throw new Error('useAuth deve ser usado dentro de <AuthProvider>');  return context;
}
```

- `getSession()` restaura a sessão guardada quando o app abre. `onAuthStateChange` mantém o estado sincronizado.
- Ao sair, `queryClient.clear()` apaga o cache: pedidos de uma conta nunca aparecem na outra.

9.2 Carrinho persistente

Código 48 · src/context/CartContext.tsx TSX

```
import AsyncStorage from '@react-native-async-storage/async-storage';
import {
 createContext,
 useCallback,
 useContext,
 useEffect,
 useMemo,
 useState,
} from 'react';
import type { ReactNode } from 'react';
import type { Cart, Product, Store } from '@/types';
import {
 EMPTY_CART,
 addToCart,
 cartTotals,
 isValidCart,
 itemCount,
 removeItem,
 replaceCart,
 setQuantity,
} from '@/utils/cart';
import { useAuth } from './AuthContext';

type CartStore = Pick<Store, 'id' | 'name' | 'delivery_fee' | 'min_order'>;
type CartProduct = Pick<Product, 'id' | 'name' | 'emoji' | 'price'>;

interface CartContextValue {
 cart: Cart;
 count: number;
 totals: ReturnType<typeof cartTotals>;
 add: (
 store: CartStore,
 product: CartProduct,
 quantity?: number,
 ) => 'added' | 'conflict';
```



```
 replaceWith: (store: CartStore, product: CartProduct, quantity?: number) => void;  setCart: (cart: Cart)
=> void;
 changeQuantity: (productId: string, quantity: number) => void;
 remove: (productId: string) => void;
 clear: () => void;
}

 const CartContext = createContext<CartContextValue | undefined>(undefined);

// Um carrinho por usuário, guardado no aparelho: sobrevive ao fechar o app. export function
CartProvider({ children }: { children: ReactNode }) {  const { user } = useAuth();
 const userId = user?.id ?? null;
 const [cart, setCartState] = useState<Cart>(EMPTY_CART);
 const [loadedFor, setLoadedFor] = useState<string | null>(null);
 const key = userId ? `marketgo.cart.${userId}` : null;

 // Carrega o carrinho do usuário logado (e descarta dados com formato inesperado).  useEffect(() => {
 let cancelled = false;
 setLoadedFor(null);
 setCartState(EMPTY_CART);
 if (!key) return;
 AsyncStorage.getItem(key)
 .then((raw) => {
 const parsed: unknown = raw ? JSON.parse(raw) : null;
 if (!cancelled && isValidCart(parsed)) setCartState(parsed);  })
 .catch(() => undefined)
 .finally(() => {
```

```
if (!cancelled) setLoadedFor(key);
});
return () => {
cancelled = true;
};
}, [key]);

// Salva a cada mudança, mas só depois de carregar (para não apagar o que estava salvo).  useEffect(() => {
if (key && loadedFor === key)
AsyncStorage.setItem(key, JSON.stringify(cart)).catch(() => undefined);  }, [cart, key,
loadedFor]);

const add = useCallback(
(store: CartStore, product: CartProduct, quantity = 1) => {
const result = addToCart(cart, store, product, quantity);
if (result.conflict) return 'conflict' as const;
setCartState(result.cart);
return 'added' as const;
},
[cart],
);

const value = useMemo<CartContextValue>(
() => ({
cart,
count: itemCount(cart),
totals: cartTotals(cart),
add,
replaceWith: (store, product, quantity = 1) =>
setCartState(replaceCart(store, product, quantity)),
setCart: setCartState,
changeQuantity: (productId, quantity) =>
setCartState((c) => setQuantity(c, productId, quantity)),
remove: (productId) => setCartState((c) => removeItem(c, productId)),  clear: () =>
setCartState(EMPTY_CART),
}),
[cart, add],
);
```



```
return <CartContext.Provider value={value}>{children}</CartContext.Provider>; }

export function useCart(): CartContextValue {
const context = useContext(CartContext);
if (!context) throw new Error('useCart deve ser usado dentro de <CartProvider>');  return context;
}
```

- **Um carrinho por usuário:** a chave de armazenamento inclui o id do usuário (`marketgo.cart.<id>`). Ao trocar de conta, o carrinho da outra conta não aparece.
- **Carregar com segurança:** o texto salvo passa por `JSON.parse` e por `isValidCart`. Se estiver quebrado, é descartado, e o app segue com o carrinho vazio.
- **Salvar só depois de carregar** (`loadedFor`). Sem essa trava, o carrinho vazio inicial apagaria o que estava salvo antes de ser lido.
- `add` devolve `'added'` ou `'conflict'`, refletindo a regra de uma loja por vez. As telas decidem o que perguntar ao usuário.
- As regras (somar, limitar, remover) **não estão aqui**: o contexto só chama as funções puras testadas da Etapa 6.

9.3 Adicionar ao carrinho com confirmação

Código 49 · src/hooks/useAddToCart.ts TypeScript

```
import { useCallback } from 'react';
import { Alert } from 'react-native';
import { useCart } from '@/context/CartContext';
import type { Product, Store } from '@/types';

type CartStore = Pick<Store, 'id' | 'name' | 'delivery_fee' | 'min_order' | 'is_open'>; type CartProduct =
Pick<Product, 'id' | 'name' | 'emoji' | 'price'>;

// Adiciona ao carrinho tratando os dois casos especiais: loja fechada e troca de loja. export function
useAddToCart() {
 const { cart, add, replaceWith } = useCart();

 return useCallback(
 (store: CartStore, product: CartProduct, quantity = 1, onAdded?: () => void) => {  if (!store.is_open) {
 Alert.alert('Loja fechada', 'Esta loja não está aceitando pedidos agora.');  return;
 }
 if (add(store, product, quantity) === 'added') {
 onAdded?.();
 return;
 }
 Alert.alert(
 'Trocar de loja?',
 `Seu carrinho tem itens de ${cart.storeName}. ` +
 `Deseja esvaziá-lo e começar um pedido em ${store.name}?`,
 [
 { text: 'Cancelar', style: 'cancel' },
 {
 text: 'Esvaziar e adicionar',
 style: 'destructive',
 onPress: () => {
```



```
 replaceWith(store, product, quantity);
 onAdded?.();
 },
 },
 ],
 );
 },
 [add, replaceWith, cart.storeName],
 );
}
```

Este hook reúne o que as telas de loja e de produto têm em comum: se a loja está **fechada**, avisa; se há **conflito de loja**, pergunta se o usuário quer esvaziar o carrinho e começar um novo pedido; se tudo certo, adiciona e executa um retorno (por exemplo, voltar à tela anterior).

Terminal bash
```
npm run typecheck
```

PONTO DE VERIFICAÇÃO

- Nenhum erro de tipo.

Etapa 10 — Navegação e autenticação

ATENÇÃO

Primeiro passo: apague o arquivo temporário app/index.tsx criado na Etapa 2. Se ele continuar, haverá duas telas para a rota /.

10.1 Como o Expo Router organiza as rotas

No Expo Router, **cada arquivo** dentro de app/ **é uma rota**. Pastas entre parênteses, como (auth), são **grupos**: organizam o código sem aparecer na URL.

Arquivo	Rota	Papel
app/_layout.tsx	—	Raiz: provedores (Query, sessão e carrinho) e a pilha
app/(auth)/login.tsx, register.tsx	/login, /register	Login e cadastro

app/(app)/_layout.tsx	—	Guarda: sem sessão, vai para /login
app/(app)/(tabs)/index.tsx	/	Lojas
app/(app)/(tabs)/cart.tsx	/cart	Carrinho
app/(app)/(tabs)/orders.tsx	/orders	Pedidos
app/(app)/(tabs)/profile.tsx	/profile	Perfil
app/(app)/store/[id].tsx	/store/123	Loja (o [id] é dinâmico)
app/(app)/product/[id].tsx	/product/123	Produto
app/(app)/checkout.tsx	/checkout	Finalizar pedido
app/(app)/order/[id].tsx	/order/123	Acompanhar pedido

10.2 Layout raiz e grupo (auth)

Código 50 · app/_layout.tsx TSX

```tsx
import { QueryClientProvider } from '@tanstack/react-query';
import { Stack } from 'expo-router';
import { StatusBar } from 'expo-status-bar';
import { AuthProvider } from '@/context/AuthContext';
import { CartProvider } from '@/context/CartContext';
import { queryClient } from '@/lib/queryClient';

export default function RootLayout() {
 return (
 <QueryClientProvider client={queryClient}>
 <AuthProvider>
 <CartProvider>
 <StatusBar style="dark" />
 <Stack screenOptions={{ headerShown: false }} />
 </CartProvider>
 </AuthProvider>
 </QueryClientProvider>
 );
}
```


Repare na **ordem** dos provedores: o CartProvider fica **dentro** do AuthProvider, porque o carrinho precisa saber quem é o usuário.

Código 51 · app/(auth)/_layout.tsx TSX

```tsx
import { Redirect, Stack } from 'expo-router';
import { LoadingView } from '@/components/StateViews';
import { useAuth } from '@/context/AuthContext';

export default function AuthLayout() {
 const { session, loading } = useAuth();

 if (loading) return <LoadingView />;
 if (session) return <Redirect href="/" />; // já logado: vai para o app

 return <Stack screenOptions={{ headerShown: false }} />;
}
```

O layout é o **guarda** do grupo: se já existe sessão, o `Redirect` envia o usuário para `/`. Enquanto a sessão é restaurada, mostramos um spinner.

Código 52 · app/(auth)/login.tsx TSX

```
import { Ionicons } from '@expo/vector-icons';
import { zodResolver } from '@hookform/resolvers/zod';
import { Link } from 'expo-router';
import { useState } from 'react';
import { useForm } from 'react-hook-form';
import { StyleSheet, Text, View } from 'react-native';
import { Button } from '@/components/Button';
import { FormInput } from '@/components/FormInput';
import { Screen } from '@/components/Screen';
import { useAuth } from '@/context/AuthContext';
import { loginSchema } from '@/schemas/auth';
import type { LoginData } from '@/schemas/auth';
import { colors, spacing } from '@/theme';
import { friendlyError } from '@/utils/errors';

export default function LoginScreen() {
 const { signIn } = useAuth();
 const [formError, setFormError] = useState<string | null>(null);

 const {
 control,
 handleSubmit,
 formState: { isSubmitting },
 } = useForm<LoginData>({
 resolver: zodResolver(loginSchema),
 defaultValues: { email: '', password: '' },
 });

 const onSubmit = handleSubmit(async ({ email, password }) => {
 setFormError(null);
 try {
 await signIn(email.trim(), password);
 // O redirecionamento acontece sozinho quando a sessão muda.
 } catch (error) {
 setFormError(friendlyError(error));
 }
 });

 return (
 <Screen scroll>
 <View style={styles.header}>
```



```
 <Ionicons name="basket" size={56} color={colors.primary} />
 <Text style={styles.title}>MarketGo</Text>
 <Text style={styles.subtitle}>Suas compras, na porta de casa</Text>  </View>

 <FormInput
 control={control}
 name="email"
 label="E-mail"
 placeholder="voce@email.com"
 keyboardType="email-address"
 autoCapitalize="none"
 autoComplete="email"
 />
 <FormInput
 control={control}
 name="password"
 label="Senha"
```

```
placeholder="Sua senha"
secureTextEntry
autoCapitalize="none"
/>

{formError ? <Text style={styles.error}>{formError}</Text> : null}  <Button title="Entrar"

onPress={onSubmit} loading={isSubmitting} />

<Link href="/register" style={styles.link}>
Não tem conta? <Text style={styles.linkStrong}>Cadastre-se</Text>  </Link>
</Screen>
);
}

const styles = StyleSheet.create({
header: { alignItems: 'center', marginTop: spacing.xl, marginBottom: spacing.xl },  title: { fontSize: 32,
fontWeight: '800', color: colors.text, marginTop: spacing.sm },  subtitle: { fontSize: 15, color: colors.muted,
marginTop: 4 },
error: { color: colors.danger, marginBottom: spacing.md, textAlign: 'center' },  link: { marginTop: spacing.lg,
textAlign: 'center', color: colors.muted, fontSize: 15 },  linkStrong: { color: colors.primary, fontWeight: '700' },
});
```

Código 53 · app/(auth)/register.tsx TSX

```
import { zodResolver } from '@hookform/resolvers/zod';
import { Link, useRouter } from 'expo-router';
import { useState } from 'react';
import { useForm } from 'react-hook-form';
import { Alert, StyleSheet, Text } from 'react-native';
import { Button } from '@/components/Button';
import { FormInput } from '@/components/FormInput';
import { Screen } from '@/components/Screen';
import { useAuth } from '@/context/AuthContext';
import { registerSchema } from '@/schemas/auth';
import type { RegisterData } from '@/schemas/auth';
import { colors, spacing } from '@/theme';
import { friendlyError } from '@/utils/errors';

export default function RegisterScreen() {
const { signUp } = useAuth();
const router = useRouter();
const [formError, setFormError] = useState<string | null>(null);

const {
control,
handleSubmit,
```



```
formState: { isSubmitting },
} = useForm<RegisterData>({
resolver: zodResolver(registerSchema),
defaultValues: { name: '', email: '', password: '', confirm: '' },  });

const onSubmit = handleSubmit(async ({ name, email, password }) => {
setFormError(null);
try {
const { needsConfirmation } = await signUp(name.trim(), email.trim(), password);  if (needsConfirmation) {
Alert.alert(
'Confirme seu e-mail',
'Enviamos um link de confirmação. Depois de confirmar, faça login.',  );
router.replace('/login');
}
// Sem confirmação: a sessão já existe e o app redireciona sozinho.  } catch (error) {
setFormError(friendlyError(error));
}
```

```
});

return (
<Screen scroll>
<Text style={styles.title}>Criar conta</Text>
<Text style={styles.subtitle}>Leva menos de um minuto.</Text>

<FormInput control={control} name="name" label="Nome" placeholder="Seu nome" /> <FormInput
control={control}
name="email"
label="E-mail"
placeholder="voce@email.com"
keyboardType="email-address"
autoCapitalize="none"
autoComplete="email"
/>
<FormInput
control={control}
name="password"
label="Senha"
placeholder="Mínimo de 6 caracteres"
secureTextEntry
autoCapitalize="none"
/>
<FormInput
control={control}
name="confirm"
label="Confirmar senha"
placeholder="Repita a senha"
secureTextEntry
autoCapitalize="none"
/>

{formError ? <Text style={styles.error}>{formError}</Text> : null} <Button title="Criar conta"

onPress={onSubmit} loading={isSubmitting} />

<Link href="/login" style={styles.link}>
Já tem conta? <Text style={styles.linkStrong}>Entrar</Text>
</Link>
</Screen>
);
}

const styles = StyleSheet.create({
title: { fontSize: 28, fontWeight: '800', color: colors.text, marginTop: spacing.lg }, subtitle: { fontSize: 15, color:
colors.muted, marginBottom: spacing.lg, marginTop: 4 },
```



```
error: { color: colors.danger, marginBottom: spacing.md, textAlign: 'center' }, link: { marginTop: spacing.lg,
textAlign: 'center', color: colors.muted, fontSize: 15 }, linkStrong: { color: colors.primary, fontWeight: '700' },
});
```

• O formulário usa `useForm` com `zodResolver`: as regras do schema viram mensagens sob cada campo. • Após o login não há `router.push`: quando a sessão muda, o guarda redireciona sozinho.

10.3 Grupo (app): área protegida e abas

Código 54 · app/(app)/_layout.tsx TSX

```
import { Redirect, Stack } from 'expo-router';
import { LoadingView } from '@/components/StateViews';
import { useAuth } from '@/context/AuthContext';
import { colors } from '@/theme';

export default function AppLayout() {
 const { session, loading } = useAuth();

 if (loading) return <LoadingView />;
 if (!session) return <Redirect href="/login" />; // sem sessão: vai para o login

 return (
 <Stack
 screenOptions={{
 headerTintColor: colors.primary,
 headerShadowVisible: false,
 headerBackButtonDisplayMode: 'minimal',
 headerStyle: { backgroundColor: colors.background },
 }}
 >
 <Stack.Screen name="(tabs)" options={{ headerShown: false }} />
 <Stack.Screen name="store/[id]" options={{ title: '' }} />
 <Stack.Screen name="product/[id]" options={{ title: 'Produto' }} />
 <Stack.Screen name="checkout" options={{ title: 'Finalizar pedido' }} />  <Stack.Screen
name="order/[id]" options={{ title: 'Acompanhar pedido' }} />  </Stack>
 );
}
```

É o guarda do outro lado: **sem sessão**, o usuário vai para `/login`. As abas ficam em `(tabs)`, e as telas de loja, produto, checkout e pedido são empilhadas por cima delas.

Código 55 · app/(app)/(tabs)/_layout.tsx TSX

```
import { Ionicons } from '@expo/vector-icons';
import { Tabs } from 'expo-router';
import type { ColorValue } from 'react-native';
import { useCart } from '@/context/CartContext';
import { colors } from '@/theme';
import type { IconName } from '@/utils/icons';

function tabIcon(name: IconName) {
 return ({ color, size }: { color: ColorValue; size: number }) => (
 <Ionicons name={name} size={size} color={color} />
 );
}

export default function TabsLayout() {
```



```
 const { count } = useCart();

 return (
 <Tabs
 screenOptions={{
 headerShown: false,
 tabBarActiveTintColor: colors.primary,
 tabBarInactiveTintColor: colors.muted,
 }}
 >
 <Tabs.Screen
 name="index"
 options={{ title: 'Lojas', tabBarIcon: tabIcon('storefront') }}
 />
 <Tabs.Screen
 name="cart"
```

```
options={{
title: 'Carrinho',
tabBarIcon: tabIcon('cart'),
tabBarBadge: count > 0 ? count : undefined,
tabBarBadgeStyle: { backgroundColor: colors.primary },
}}
/>
<Tabs.Screen
name="orders"
options={{ title: 'Pedidos', tabBarIcon: tabIcon('receipt') }}
/>
<Tabs.Screen
name="profile"
options={{ title: 'Perfil', tabBarIcon: tabIcon('person') }}
/>
</Tabs>
);
}
```

A aba **Carrinho** exibe um selo com a quantidade de itens (`tabBarBadge`), lida do contexto do carrinho.

Código 56 · app/(app)/(tabs)/profile.tsx TSX

```
import Constants from 'expo-constants';
import { Alert, StyleSheet, Text, View } from 'react-native';
import { Button } from '@/components/Button';
import { Card } from '@/components/Card';
import { Screen } from '@/components/Screen';
import { useAuth } from '@/context/AuthContext';
import { colors, spacing } from '@/theme';
import { friendlyError } from '@/utils/errors';

export default function ProfileScreen() {
const { user, signOut } = useAuth();
const name = String(user?.user_metadata?.full_name ?? 'Usuário');

function confirmSignOut() {
Alert.alert('Sair', 'Deseja encerrar a sessão?', [
{ text: 'Cancelar', style: 'cancel' },
{
text: 'Sair',
style: 'destructive',
onPress: () => signOut().catch((e) => Alert.alert('Erro', friendlyError(e))),  },
]);
}

return (
<Screen scroll>
<Text style={styles.title}>Perfil</Text>
```



```
<Card>
<View style={styles.avatar}>
<Text style={styles.avatarText}>{name.charAt(0).toUpperCase()}</Text>  </View>
<Text style={styles.name}>{name}</Text>
<Text style={styles.email}>{user?.email}</Text>
</Card>

<Card>
<Text style={styles.infoTitle}>Sobre este app</Text>
<Text style={styles.infoText}>
Projeto de estudo. Os pagamentos são simulados e nenhum pedido é entregue de  verdade. O carrinho
fica salvo no seu aparelho.
```

```
</Text>
</Card>

<Button title="Sair da conta" variant="danger" onPress={confirmSignOut} />  <Text
style={styles.version}>
 MarketGo v{Constants.expoConfig?.version ?? '1.0.0'}
 </Text>
 </Screen>
 );
}

 const styles = StyleSheet.create({
 title: {
 fontSize: 26,
 fontWeight: '800',
 color: colors.text,
 marginBottom: spacing.md,
 },
 avatar: {
 width: 72,
 height: 72,
 borderRadius: 36,
 backgroundColor: colors.primary,
 alignItems: 'center',
 justifyContent: 'center',
 alignSelf: 'center',
 marginBottom: spacing.sm,
 },
 avatarText: { color: '#FFFFFF', fontSize: 30, fontWeight: '800' },
 name: { fontSize: 20, fontWeight: '700', color: colors.text, textAlign: 'center' },  email: { fontSize: 14, color:
colors.muted, textAlign: 'center', marginTop: 2 },  infoTitle: { fontSize: 15, fontWeight: '700', color:
colors.text, marginBottom: 4 },  infoText: { fontSize: 14, color: colors.muted, lineHeight: 20 },
 version: {
 textAlign: 'center',
 color: colors.muted,
 fontSize: 12,
 marginTop: spacing.lg,
 },
});
```

10.4 Telas provisórias

Para navegar já nesta etapa, crie os **sete arquivos** abaixo com o mesmo conteúdo provisório. Nas próximas etapas, eles serão substituídos pelas telas reais.

- app/(app)/(tabs)/index.tsx
- app/(app)/(tabs)/cart.tsx
- app/(app)/(tabs)/orders.tsx
- app/(app)/store/[id].tsx

- app/(app)/product/[id].tsx
- app/(app)/checkout.tsx
- app/(app)/order/[id].tsx

Código 57 · Conteúdo provisório dos sete arquivos TSX

```
import { Text, View } from 'react-native';

export default function Placeholder() {
 return (
```

```
<View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>  <Text>Em construção</Text>
</View>
);
}
```

Terminal bash npx expo start -c

PONTO DE VERIFICAÇÃO

- O app abre na tela de **login**.
- No cadastro, uma senha de 3 caracteres mostra “Use pelo menos 6 caracteres”. • Um cadastro válido leva ao app com **quatro abas**: Lojas, Carrinho, Pedidos e Perfil. • Feche o app por completo e reabra: você continua logado.
- Em **Perfil**, aparecem seu nome e e-mail. **Sair da conta** leva de volta ao login. • No Supabase, em **Authentication** → **Users**, o seu usuário está listado.

ATENÇÃO

Se aparecer “Confirme seu e-mail antes de entrar”, volte à seção 3.4 e desligue a confirmação de e-mail.

Etapa 11 — Catálogo: lojas, cardápio e produto

11.1 Barra do carrinho

Código 58 · src/components/CartBar.tsx TSX

```tsx
import { useRouter } from 'expo-router';
import { Pressable, StyleSheet, Text } from 'react-native';
import { useCart } from '@/context/CartContext';
import { colors, radius, shadow, spacing } from '@/theme';
import { formatMoney } from '@/utils/money';

// Barra fixa com o resumo do carrinho, para ir direto ao pedido.
export function CartBar() {
 const router = useRouter();
 const { count, totals } = useCart();
 if (count === 0) return null;

 return (
 <Pressable
 accessibilityRole="button"
 onPress={() => router.push('/cart')}
 style={({ pressed }) => [styles.bar, pressed && styles.pressed]}
 >
 <Text style={styles.text}>
 Ver carrinho · {count} {count === 1 ? 'item' : 'itens'}
 </Text>
 <Text style={styles.total}>{formatMoney(totals.subtotal)}</Text>
 </Pressable>
 );
}

const styles = StyleSheet.create({
 bar: {
 position: 'absolute',
 left: spacing.md,
 right: spacing.md,
 bottom: spacing.md,
 minHeight: 56,
 borderRadius: radius.lg,
 backgroundColor: colors.primary,
 paddingHorizontal: spacing.lg,
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 ...shadow,
 },
 pressed: { opacity: 0.9 },
 text: { color: '#FFFFFF', fontSize: 16, fontWeight: '700' },
 total: { color: '#FFFFFF', fontSize: 16, fontWeight: '800' },
});
```

Barra fixa no rodapé com o resumo do carrinho. Ela só aparece quando há itens e leva à aba do carrinho.

11.2 Lista de lojas

Código 59 · app/(app)/(tabs)/index.tsx TSX

```tsx
import { useRouter } from 'expo-router';
import { useMemo, useState } from 'react';
import {
 FlatList,
 RefreshControl,
 StyleSheet,
 Text,
 TextInput,
 View,
```

```
} from 'react-native';
import { CategoryChips } from '@/components/CategoryChips';
import { Screen } from '@/components/Screen';
import { EmptyState, ErrorState, LoadingView } from '@/components/StateViews'; import {
StoreCard } from '@/components/StoreCard';
import { useAuth } from '@/context/AuthContext';
import { useStores } from '@/hooks/useCatalog';
import { colors, radius, spacing } from '@/theme';
import type { StoreCategory } from '@/types';
import { filterStores, sortStores } from '@/utils/catalog';
import { friendlyError } from '@/utils/errors';

export default function StoresScreen() {
 const router = useRouter();
 const { user } = useAuth();
 const { data, isLoading, isError, error, refetch, isRefetching } = useStores();  const [query, setQuery]
= useState('');
 const [category, setCategory] = useState<StoreCategory | null>(null);

 const stores = useMemo(
 () => sortStores(filterStores(data ?? [], query, category)),
 [data, query, category],
 );
 const firstName = String(user?.user_metadata?.full_name ?? '').split(' ')[0];

 if (isLoading) return <LoadingView />;
 if (isError) return <ErrorState message={friendlyError(error)} onRetry={refetch} />;

 const header = (
 <View style={styles.header}>
 <Text style={styles.greeting}>Olá{firstName ? `, ${firstName}` : ''} ??</Text>  <Text
style={styles.title}>O que vai pedir hoje?</Text>
 <TextInput
 value={query}
 onChangeText={setQuery}
 placeholder="Buscar loja ou tipo de comida"
 placeholderTextColor={colors.muted}
 style={styles.search}
 clearButtonMode="while-editing"
 />
 <CategoryChips value={category} onChange={setCategory} />
 </View>
 );

 return (
 <Screen>
 <FlatList
 data={stores}
 keyExtractor={(item) => item.id}
 contentContainerStyle={styles.content}
 ListHeaderComponent={header}
 ListEmptyComponent={
 <EmptyState
 icon="search-outline"
 title="Nenhuma loja encontrada"
 message="Tente outra busca ou outra categoria."
 />
 }
 renderItem={({ item }) => (
 <StoreCard store={item} onPress={() => router.push(`/store/${item.id}`)} />  )}
 keyboardShouldPersistTaps="handled"
 refreshControl={<RefreshControl refreshing={isRefetching} onRefresh={refetch} />}  />
 </Screen>
 );
```



```
}
```

```
const styles = StyleSheet.create({
 content: { padding: spacing.md, flexGrow: 1 },
 header: { marginBottom: spacing.sm },
 greeting: { fontSize: 15, color: colors.muted },
 title: {
 fontSize: 26,
 fontWeight: '800',
 color: colors.text,
 marginBottom: spacing.md,
 },
 search: {
 minHeight: 48,
 borderWidth: 1,
 borderColor: colors.border,
 borderRadius: radius.md,
 backgroundColor: colors.surface,
 paddingHorizontal: spacing.md,
 fontSize: 15,
 color: colors.text,
 },
});
```

- `useMemo` combina busca, categoria e ordenação. A busca é uma função **pura** (testada na Etapa 6), então responde na hora, sem rede.
- O cabeçalho (saudação, busca e categorias) é passado como **elemento** e não como componente, para o campo de busca não perder o foco a cada letra digitada.
- Puxar a lista para baixo (pull to refresh) refaz a consulta.

11.3 Cardápio da loja

Código 60 · app/(app)/store/[id].tsx TSX

```
import { Stack, useLocalSearchParams, useRouter } from 'expo-router';
import { useMemo } from 'react';
import { SectionList, StyleSheet, Text, View } from 'react-native';
import { CartBar } from '@/components/CartBar';
import { Screen } from '@/components/Screen';
import { ErrorState, LoadingView } from '@/components/StateViews';
import { ProductRow } from '@/components/ProductRow';
import { useCart } from '@/context/CartContext';
import { useAddToCart } from '@/hooks/useAddToCart';
import { useProducts, useStore } from '@/hooks/useCatalog';
import { colors, radius, spacing } from '@/theme';
import { formatEta, groupProducts } from '@/utils/catalog';
import { quantityOf } from '@/utils/cart';
import { friendlyError } from '@/utils/errors';
import { formatMoney } from '@/utils/money';

export default function StoreScreen() {
 const { id } = useLocalSearchParams<{ id: string }>();
 const router = useRouter();
 const store = useStore(id);
 const products = useProducts(id);
 const { cart } = useCart();
 const addToCart = useAddToCart();

 const sections = useMemo(() => groupProducts(products.data ?? []), [products.data]);  if (store.isLoading

|| products.isLoading) return <LoadingView />;
```

```
if (store.isError || products.isError || !store.data) {
return (
<ErrorState
message={friendlyError(store.error ?? products.error)}
onRetry={() => {
store.refetch();
products.refetch();
}}
/>
);
}

const s = store.data;
const header = (
<View style={styles.header}>
<View style={styles.headerTop}>
<View style={styles.logo}>
<Text style={styles.logoEmoji}>{s.emoji}</Text>
</View>
<View style={styles.headerInfo}>
<Text style={styles.name}>{s.name}</Text>
<Text style={styles.meta}>
{s.category} · ⭐ {s.rating.toFixed(1).replace('.', ',')}
</Text>
</View>
</View>
<Text style={styles.description}>{s.description}</Text>
{s.is_open ? (
<Text style={styles.facts}>
{formatEta(s.eta_min, s.eta_max)} · Entrega{' '}
{s.delivery_fee === 0 ? 'grátis' : formatMoney(s.delivery_fee)} · Pedido mínimo{' '} {formatMoney(s.min_order)}
</Text>
) : (
<View style={styles.closed}>
<Text style={styles.closedText}>
Fechada agora. Você pode ver o cardápio, mas não pedir.
</Text>
</View>
)}
</View>
);

return (
<Screen edges={['bottom', 'left', 'right']}>
<Stack.Screen options={{ title: s.name }} />
<SectionList
sections={sections}
keyExtractor={(item) => item.id}
ListHeaderComponent={header}
contentContainerStyle={styles.content}
renderSectionHeader={({ section }) => (
<Text style={styles.section}>{section.title}</Text>
)}
renderItem={({ item }) => (
<ProductRow
product={item}
quantityInCart={cart.storeId === s.id ? quantityOf(cart, item.id) : 0} disabled={!s.is_open}
onPress={() => router.push(`/product/${item.id}`)}
onQuickAdd={() => addToCart(s, item)}
/>
)}
stickySectionHeadersEnabled={false}
/>
<CartBar />
</Screen>
);
}
```